

RP-09 — Final Synthesis & Execution Roadmap: Group Lesson Planner

Metadata

Field	Value
Report id	RP-09
Date	2026-08-20
Subject	One sequenced, executable plan for a solo developer: from the live single-file app to a migrated, durably persisted, improved application
Inputs consumed	The research agent contract; the verified-facts addendum; RP-01 through RP-08 in full (docs/research/rp01-app-inventory/ ... docs/research/rp08-ux-a11y-audit/)
Nothing runtime-verified here	This report ran no browser, no build, no test and no network fetch. Every behavioural, numeric and vendor fact is carried from a named input report and attributed there
Orchestrator-verified inputs treated as settled	GitHub Pages publishing source is "Deploy from a branch", main / (root) ; AGENTS.md documents npm scripts that exist in no tracked file; Firestore has no scheduled backup, no PITR and no managed export on the free plan; Firebase sign-in must use signInWithPopup ; the "API keys are not secrets" claim is conditional; GDPR Art. 30(5) is not a blanket exemption; RP-08's four corrections on WCAG version, the clipped month button, navigator.storage.estimate() and performance-versus-layout
Personal-data check	Verified by scan over this report: zero IBAN-shaped strings, zero Cyrillic characters, and the only runs of eight or more digits are two document identifiers inside source URLs — a Google support-article id in S30 and a EUR-Lex CELEX identifier in S34. The words IBAN and MFO appear only as the literals the repository guard is instructed to match, never as values. The template's contents are referred to by line reference and category only
Verification statement	Reconciliations in §2 are reasoned from the eight inputs plus the addendum , with the deciding evidence named in each row. External sources in §11 are those the <i>conclusions</i> depend on, carried forward from the input reports' own source tables rather than re-fetched. Line citations use the 1473-line committed index.html at commit 211eb6b , per RP-01's baseline — see the standing warning under §9
Not verified	The end user's browser, device and OS; the conformance level of WCAG 2.2 SC 2.5.8; anything requiring a live Firebase project, a build, or a test run

1. Executive summary

- **Order of operations is fixed and non-negotiable:** dump the raw storage on every surviving device today; remove the published personal payment identifiers; guard the storage layer; ship a versioned JSON backup with a staleness nag. Nothing else starts before these.

- **Storage hardening and the JSON backup land in the legacy `index.html`** . Cloud sync ships **once**, in the React app, after cutover — pulled forward only if the staleness indicator crosses 14 days twice.
- **The durable store is Firestore `users/{uid}` on Spark in `europa-central2` , plus client-rotated snapshots at `users/{uid}/snapshots/{ts}` , plus the local JSON envelope**. Firestore alone is replication, never backup.
- **Auth is Google-provider only, `signInWithPopup` from a visible button**, `denyslystopadskyy.github.io` on the authorized-domain list, rule `request.auth.uid == uid` . Never `signInWithRedirect` , never anonymous auth.
- **Migration follows RP-02 Strategy C:** Vite + React + TypeScript, no router, `base: '/lesson-planner/'` , staging at `/lesson-planner/next/` writing **prefixed** storage keys. RP-02's P3 exit criterion is amended in §2 accordingly.
- **Land the diverged working copy of `index.html` , commit the toolchain, freeze the eight `data-testid` names as a contract React must reproduce, cap the DOM-level suite at its current size, drop the 17-file screenplay layer**, and put every new test hour into the storage-contract layer.
- **Test layers kept:** Playwright E2E on Chromium only, storage contract, and Vitest pure-function units. **Dropped:** jsdom component tests, pixel visual regression, cross-browser matrix, mutation testing, coverage thresholds.
- **The accessibility target is WCAG 2.2 Level AA**. Target size passes; the 44×44 figure is Level AAA and out of scope. Remediation lands inside the React components, ranked fourth, not as patches to the legacy file.
- **Switching the Pages publishing source to GitHub Actions is a manual repository-owner action and a hard prerequisite for the build step**. Never select `main / /docs` .
- **Never build:** routing, payments, automated sending, student records, PDF invoices, portals, an i18n framework, CRDTs, Postgres, a subscribable calendar feed, or client-side encryption of group names.

2. Resolved conflicts

#	Conflict	Resolution	Reasoning	Confidence
C1	RP-05 says local hardening plus JSON backup "ships first"; RP-02's phasing builds React beside the live app and says extract "by copy", leaving <code>index.html</code> untouched	Storage hardening and the JSON backup land in the legacy <code>index.html</code> , committed and deployed, before any React work	RP-02's own P0 already includes "land the wording and privacy fixes", and its don't-touch rule applies to the <i>extraction</i> (§6: do it by copy), not to bug fixes. So the conflict dissolves rather than needing adjudication. Decisive tiebreaker: the app has already lost all its data once, RP-01 D14 is runtime-verified as silent and total, and P3 is the first React artifact a user could touch — many evenings away	High

#	Conflict	Resolution	Reasoning	Confidence
C2	RP-05 §7 offers a no-build CDN path so Firestore sync could ship into the legacy file; RP-02 wants one app	Sync ships once, in React, after cutover. The CDN path is retained only as the documented fallback if the migration stalls and the trigger below fires	The try/catch guard is 26 lines and portable, so doing it twice is cheap. Sync is the first third-party dependency, needs the personal-data cleanup, the auth mechanics and the snapshot layer — doing that twice is expensive. With a JSON backup plus a staleness nag in place, sync's marginal urgency drops from critical to high. Pull-forward trigger: the staleness indicator reaches 14 days twice, i.e. she demonstrably will not press the button	High
C3	RP-03: the e2e suite cannot pass against the deployed page because its locators need <code>data-*</code> present only in the uncommitted copy. RP-02: DOM-level pinning is near-worthless across the migration	Land the diverged copy, freeze its eight <code>data-testid</code> values as a contract React must reproduce, cap the DOM suite at its current size, drop the screenplay layer, and direct all new test investment to the storage-contract layer. Delete the legacy DOM specs after cutover	This deliberately triggers RP-02's own F7 — the only condition under which DOM hooks stop being sunk cost — rather than contradicting RP-02. RP-03's fix is an 18-line diff that also closes RP-01 D8 and D9, and it makes 1,846 already-written lines runnable instead of dead. RP-02 is still honoured: no <i>growth</i> in DOM assertions, and [S18]'s delete-afterwards rule applies at R13	High
C4	RP-05 recommends Firestore as the durable store; RP-07 plus the addendum establish that Spark has no scheduled backup, no PITR and no managed export	One combined recommendation: Firestore <code>users/{uid}</code> (replication) + client-rotated snapshot documents <code>users/{uid}/snapshots/{ts}</code> , 14 daily plus first-of-month for 12 months (rollback) + the local JSON envelope with a staleness indicator (provider-independent escape hatch). Firestore alone must never be described as a backup	Orchestrator-verified: scheduled backups require Blaze [S1], PITR requires billing [S2], managed export requires Blaze [S3]. A single mutable replica propagates a bad write everywhere with nothing to roll back to, and RP-01 documents four mechanisms by which this app corrupts its own state. RP-07's snapshot layer is therefore a required part of the RP-05 recommendation, not an optional extra	High

#	Conflict	Resolution	Reasoning	Confidence
C5	RP-05 §7 specifies <code>signInWithRedirect</code>	<code>signInWithPopup</code> , hung off a visible button (a popup needs a user gesture), with <code>denyslystopadskyy.github.io</code> added to the authorized-domain list	Orchestrator-verified against Firebase's own guidance [S4]: the redirect flow is broken for apps off the Firebase auth domain, already enforced on Firefox 109+ and Safari 16.1+. Record that the domain grant covers every GitHub Pages project on that hostname [S5]	High
C6	RP-01 records target size as passing; RP-08 records it as failing	Name the version: the target is WCAG 2.2 Level AA, and target size passes. Under 2.2 the applicable minimum is SC 2.5.8's 24×24, against RP-08's measured 41.6×29 and 79.4×29. The 44×44 figure comes from SC 2.5.5, which is Level AAA in both versions and outside an AA target	Both reports were right about their own version; RP-08 verified that 2.5.8 does not exist in 2.1 [S25]. The resolution is robust to the one thing not verified: if SC 2.5.8 turned out to be AAA rather than AA, target size would have no AA requirement under 2.2 either, so the verdict is unchanged. The 44 px work is reclassified from conformance to phone ergonomics — which is RP-08's own conclusion	High
C7	RP-03 wants a capped jsdom component layer (three components); RP-04 rejects component tests outright as duplicating Playwright	Drop the component layer. Vitest is for pure functions only. Re-adopt trigger: a component-wiring bug escapes to the deployed app twice	RP-04's argument is stronger for a solo maintainer — two suites asserting the same behaviour disagree, and the wiring RP-03 wants covered is already exercised end to end at this app's size. Both reports agree on Vitest for pure functions, which is where RP-01's D4, D12 and D20 live	Medium
C8	RP-02 P1 prefers <code>node:test</code> (zero dependency); RP-03 and RP-04 specify Vitest	Vitest, from the extraction step onward	RP-02 explicitly labelled <code>node:test</code> "a preference, not a requirement of this plan". Installing Vite plus Vitest at extraction time avoids a later migration, and Vitest peers the Vite the React build needs anyway [S26]	High

#	Conflict	Resolution	Reasoning	Confidence
C9	RP-04 §5 requires the staging build to write prefixed keys; RP-02 P3's exit criterion requires the React app to render the user's real data over the same three keys	Staging writes prefixed keys. RP-02's P3 exit criterion is amended to: "the React app, seeded from a JSON export of the live data, renders it correctly, and its writes round-trip back through the legacy app." A one-off unprefixed validation is permitted only on the developer's own machine, time-boxed, with a fresh export in hand — never on hers	The amended criterion tests the same property (read-compatible, write-identical) without a permanent shared-key arrangement. The risk is asymmetric: an unfinished React build sharing the live keys can destroy the only copy, and <code>scripts/assert-storage-keys.sh</code> then enforces unprefixed literals only in the cutover build	High
C10	RP-03 standardises the test origin on <code>http://127.0.0.1:4173</code> ; RP-04 requires <code>baseUrl</code> to carry the base path, as <code>http://localhost:4173/lesson-planner/</code>	<code>http://127.0.0.1:4173/lesson-planner/</code>	Not a real disagreement — each report changed one half. RP-03's host choice is load-bearing (one origin for <code>localStorage</code> seeding, and a secure context for the clipboard write); RP-04's path is load-bearing once <code>Vite base</code> is set. Both hold together	High
C11	RP-03's CI is advisory and cannot gate a deploy; RP-04's CI structurally gates the deploy	Sequenced, not contradictory: a test-only workflow now (R4), the artifact-deploy workflow at R8 after the publishing source is flipped	Branch publishing has no build step to gate, so a gate is impossible today. RP-04's structural gate becomes available at the same moment the flip happens	High
C12	RP-03 CI pins <code>node-version: lts/*</code> ; RP-04 pins <code>node-version: 22</code>	Pin 22	RP-04's own rejection list rules out a Node matrix; a floating <code>lts/*</code> is a silent version bump on a project checked rarely. One runtime, one deployment, one pinned major	High
C13	RP-01 rates the pluralisation defect D19 Low; RP-08 rates it Medium	Medium	RP-08 verified the identical wording is in the shipped default template at <code>index.html:384</code> , so it reaches the message students read, not only two internal UI strings	High

#	Conflict	Resolution	Reasoning	Confidence
C14	RP-01 says both clipped calendar controls are reachable by scrolling at 375 px; RP-08 measured that the previous-month button is not	RP-08 is correct — <code>scrollLeft</code> cannot go below 0 while <code>justify-content: center</code> pushes content off the left edge	Addendum, orchestrator-verified. Raises the fix from cosmetic to functional: it is the only permanently unreachable control in the app	High
C15	RP-05 cites a 10 GiB figure as storage headroom; RP-08 measured the real <code>localStorage</code> ceiling and showed <code>navigator.storage.estimate()</code> does not answer the question	Cite RP-08's direct probe: 5,177,344 characters accepted in one key, the next 64 KiB write threw. Never cite 10 GiB as the <code>localStorage</code> ceiling	Addendum. Directionally the conclusion is the same — quota is a settled non-issue at a 70 KB three-year dataset — but the number does not mean what it appears to mean, and M9b adds that the ceiling is an origin budget shared across keys, which matters when a fourth key is added	High
C16	RP-01 groups <code>updateDefaultPrice</code> and <code>handleSelectedDatesPriceChange</code> as two paths that mutate without persisting	Only <code>updateDefaultPrice</code> does. Prefer RP-02's account	Addendum. <code>handleSelectedDatesPriceChange</code> writes through <code>getActiveOverrides</code> to the draft and is reachable only while <code>isEditing</code> is true, so not persisting is correct there. This localises the "Cancel does not revert the price" and "price edit discards an unsaved name edit" defects to one missing draft, which is what makes them fixable structurally in React rather than by patching two handlers	High

#	Conflict	Resolution	Reasoning	Confidence
C17	RP-06 QW-3 wants a schedule-style placeholder on the group-name input; RP-08 QW-6 wants that placeholder to read <code>Untitled Group</code>	One field, one placeholder: a non-identifying schedule-style example. Clear the literal prefix; do not surface the fallback name in the placeholder	Both prompts edit the same attribute on the same element with different values. RP-06's minimisation measure is the higher-leverage of the two (it moves the only GDPR-triggering field from "conditional" to "none"), and RP-08's own note records that <code>render.groupInfo</code> already falls back to <code>Untitled Group</code> when the field is empty, so nothing is lost. Merged prompt reissued as QW-17	High
C18	RP-05 QW-2's backup file has four top-level members; RP-07 §2 defines a richer envelope serving both file and cloud sinks	RP-07's envelope wins, with <code>counts</code>, <code>source</code> and <code>fingerprint</code> optional until the fingerprint utility lands	RP-07 is the later design and explicitly states "one transform, two sinks", and §6 Procedure B is written against it. Adopting RP-05's narrower shape first would force a format migration inside the same programme. Merged prompt reissued as QW-8	High
C19	RP-05 QW-5's import confirmation names the existing group count; RP-07 §3 step 5.2 requires it to name both sides	Name both sides — "Replace N groups / M lessons on this device with P groups / Q lessons from this file?"	RP-07's wording is what makes the dialog a decision rather than a speed bump, and RP-01 D12 is the case where the incoming side is garbage. Prompt reissued as QW-10	High
C20	RP-02 F3 says that if Pages cannot serve a built artifact, fall back to Strategy B with committed build output	F3 does not fire. The addendum establishes the publishing source is branch-based today and that switching to GitHub Actions is a one-time owner action, so artifact deployment is available	Removes the only condition under which RP-02's recommendation flips to B, and removes any need to commit <code>dist</code> against <code>AGENTS.md:36</code>	High

3. Open decisions requiring the developer's input

#	Decision	Options	What would settle it
D1	Does the diverged working copy of <code>index.html</code> land?	(a) Land it — recommended; (b) discard it and correct the four prior-art <code>docs/*.md</code> files' line numbers instead	His call alone. Recommendation: land it. It is an 18-line diff that closes RP-01 D8 and D9, is already exercised, and is the only thing that makes 1,846 lines of written tests runnable. If discarded, correcting the docs' 8-to-18-line drift becomes mandatory, not optional
D2	Is the cross-month bulk price write intended?	(a) Intended — pin it as an invariant; (b) a defect — scope the write to the visible month; (c) neither — state the count and require confirmation above one month	A product answer from the teacher: "when you type a price with dates selected in three months, do you mean all three?" Blocks RP-08 B16, and decides whether RP-03 tags the pricing tests CHAR or DEF. RP-08 measured one keystroke rewriting 36 months at three years of history
D3	Will he switch Settings → Pages → Build and deployment to GitHub Actions ?	(a) Yes — the whole build plan proceeds; (b) no — the only branch-based route is committing built output to the root of <code>main</code> , against <code>AGENTS.md:36</code>	A decision, not a workaround. Nothing in CI can perform it. Confirm before R8 starts. Never select <code>main / /docs</code> : that would publish <code>docs/research/</code> in place of the application
D4	Should Clear All Data also clear <code>paymentTemplate</code> ?	(a) Yes — matches the confirm text and removes the key that holds personal data; (b) no — keep a customised template across a wipe	The confirm text already promises a full wipe, so (a) is both the correctness and the privacy answer. It is still a behaviour change she would notice, so it needs one sentence to her
D5	Is <code>beforeunload</code> wanted at all?	(a) Remove it — recommended once the storage guard and the backup nag exist; (b) keep it	She will notice the "leave site?" dialog stopping. One question to her. Its CSV-backup advice is never shown, because browsers discard custom text
D6	Who is the person named in the default payment template — himself or a third party?	(a) Himself; (b) the teacher or another person	Determines whether the publication was of his own identifiers or someone else's, which changes the disclosure conversation and the GDPR posture. Ask him. Either way, treat the values as permanently disclosed
D7	Should <code>defaultCurrency</code> stay a single global?	(a) Keep the global; (b) drop it and derive the new-group default from the last-edited group	Ask her whether groups legitimately differ in currency. RP-08 confirms per-group currency already works; only the default is global, and it is overwritten by the last saved group and by the first CSV row

#	Decision	Options	What would settle it
D8	Should the typed model keep the two-array denormalisation of <code>dates</code> ?	(a) Keep and enforce the invariant; (b) eliminate it, deriving <code>group.dates</code> from the overrides	An architecture decision for R6/R9. RP-07's <code>datesParity</code> check will report whether the live data has already diverged, which should inform it
D9	What retention period does she actually need?	Driven by Polish tax and accounting record-keeping rules	Her accountant. Do not guess a number, and do not automate deletion in an app with this loss history
D10	Is a service worker wanted for offline use and installability?	(a) No, for now — recommended; (b) yes, as its own decision after cutover	Genuinely valuable for a teacher on a wiped laptop, and it exempts the origin from WebKit's seven-day sweep. But it introduces a stale-build failure mode that can strand her on old code, which is the exact class of problem the cutover plan exists to avoid. Needs its own decision, not a footnote

4. Immediate-action block (data-loss and exposure risk)

Everything in this block is separated from the roadmap because it is either time-critical or a blocker on later work. Steps 1 to 4 need no code. **Nothing may be uploaded, emailed, synced or placed in a shared folder until step 5 has landed** — RP-05, RP-06 and RP-07 all say so independently, because the default template reaches every user who never edited it.

Today, in this order.

- 1. Stop the fastest clock.** On the affected machine: stop using it for anything that writes to disk, and disconnect the Time Machine disk so a fresh backup cannot age out the pre-incident snapshot. macOS local snapshots are kept roughly 24 hours; if more than a day has passed, that path is already closed.
- 2. Run the raw storage dump on every device and browser that might hold data** — second laptop, phone, tablet, work computer. It reads the three keys as raw strings and never parses them, so it survives corrupt data, and it needs no deploy. This is quick win QW-1 and it is the only action in the whole programme that can still save data that exists right now.
- 3. Search for artifacts, not bytes.** `lesson-planner-*.csv` on every device and in Downloads, mail and chat; the payment messages she already sent (they carry the month, lesson count and total per group, which reconstructs the money even if individual dates are gone); her own calendar or notebook. These two searches are more likely to pay off than anything involving browser internals.
- 4. Send her one message with four questions.** Which browser, device and operating system does she use — see §7, where one answer closes a cluster of unknowns. Did she ever click Save CSV, and where did those files go. How many groups and months did she keep. What exactly did "cleaning the laptop" mean. Do not bundle product questions into this message; it is a rescue message.

Day 1, and it blocks everything cloud-shaped.

5. **Remove the personal payment identifiers from the default template** (`index.html:387-392` and the personal first name at `400`), fixing the template's `1 lessons` wording in the same edit, then **add the repository guard** that fails when IBAN-shaped or long-digit strings reappear. Treat the identifiers as **permanently disclosed** — they have been publicly readable since the first commit — and tell the named person so they can decide about the bank account. A git-history purge is optional mitigation afterwards, never remediation, and it must not delay the conversation. Quick wins QW-2 and QW-3.

Weekend 1 — stop the next silent loss.

6. **Guard the storage layer:** per-key try/catch around both `JSON.parse` calls, the unparseable value copied to `<key>.corrupt.backup`, a shape check before assignment, guarded `setItem`, and `init()` always completing. This closes the runtime-verified dead-page failure that is the most plausible mechanism for a repeat of the original loss. QW-4.
7. **Close the three ways the app corrupts or bricks its own data:** escape the group name in the card sink (verified script execution, and a credential-theft path the moment a signed-in session exists), constrain the year and price inputs, and sanitise the currency on import. QW-5, QW-6, QW-7.

Weekend 2 — give her a backup that works.

8. **Ship the versioned JSON envelope** covering all three keys, the **fingerprint utility** that makes any restore checkable, the **pre-destructive snapshot with an undo**, the **import confirmation naming both sides**, the **last-backup age indicator with a nag**, and the **persistent-storage request**. QW-8 through QW-14.
9. **Rehearse Procedure B end to end with her once, on purpose, while nothing is broken.** An unrehearsed restore is a hope. Do not hand her a procedure whose buttons do not exist yet — that is the prior-art failure this programme already documented.

5. Sequenced roadmap

Effort bands are S / M / L, relative to each other and to one engineer's evenings. They are not hours, and no hour figure appears anywhere in this report.

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R0	Rescue what still exists, and stop the published exposure	§4 steps 1-5: raw dump on every device, artifact search, the four questions, personal-data removal plus template wording, repo guard, disclosure conversation	Every reachable device dumped; the guard script exits non-zero on the old file and zero on the new; the named person has been told	Guard script run from two working directories; grep for IBAN shape, 8+ digit runs, IBAN and MF0 over index.html returns only the permitted Ukrainian payment-purpose line	none	S
R1	Stop the next silent loss in the file that is actually deployed	Storage try/catch plus .corrupt.backup plus shape check; escape group.name at index.html:1047 ; constrain year and price inputs with handler clamps; sanitise currency on import	A planted truncated value leaves 12 month options, the empty state rendered and + Add Group working; a markup group name renders as text with no element created; a one-digit year cannot produce a short-year key; US Dollar imports to a three-letter code	Manual console checks named in QW-4 to QW-7; console error list empty in every case	R0 step 5	S

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R2	Replace "her memory" with a real backup she can restore from	JSON envelope export and import; fingerprint utility; snapshot-before-destructive plus undo; import confirmation naming both sides; last-backup age with a nag past 14 days; <code>navigator.storage.persist()</code> ; additive schema-version key with a forward-compatibility refusal	Export, clear, re-import restores all three keys byte-for-byte including the template; four destructive actions leave exactly three snapshot generations; a version of 2 puts the app read-only and writes nothing	Procedure B rehearsed with her end to end; fingerprint identical before and after a round trip; <code>Object.keys(localStorage)</code> inspected after each step	R1	M
R3	Freeze the baseline so every later citation and command is true	Land the diverged <code>index.html</code> ; re-verify the line citations in every unshipped quick-win prompt; commit <code>package.json</code> , <code>package-lock.json</code> , <code>playwright.config.ts</code> , <code>tsconfig.json</code> and <code>e2e/</code> ; extend <code>.gitignore</code> for build output and env or credential files; answer D1, D2, D4, D5, D7	Exactly one <code>index.html</code> committed and deployed, byte-identical to the served bytes; a fresh clone plus <code>npm ci</code> runs the documented commands; no unshipped prompt cites a line number that has moved; the repo guard still passes	<code>curl -sI</code> on the live URL plus a byte diff against the committed file; fresh-clone install and script run; guard script re-run after the merge	R2	S

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R4	Make the existing suite runnable, correct and capped	Replace the custom context fixture with a <code>storageState</code> option override; standardise on <code>http://127.0.0.1:4173/lesson-planner/</code> ; freeze the clock with <code>setFixedTime</code> before navigation and derive every date from it; drop the 17-file screenplay layer; retag every spec INV / DEF / CHAR; add the storage-contract spec and fixture corpus; assert CSV export content and a full round trip; add the test-only CI workflow; delete the orphaned visual-snapshot directory	<code>npm run typecheck</code> and the suite both pass from a fresh clone; no <code>browser.newCon</code> text and no <code>faker.date</code> under <code>e2e/</code> ; two runs on different calendar days produce identical months; every spec carries a tag; CI runs on every pull request	Run the suite twice on different dates and diff the resolved months; deliberately fail one assertion and confirm trace, screenshot and video are all attached	R3	M
R5	Ship the legacy fixes that pay back every month, and stop money going wrong quietly	Month rows newest-first; calendar controls wrap and the summary stops clipping; group-name field cleared with a non-identifying placeholder; group list refreshed after a schedule save; the two UI count strings pluralised; month in each copy button's accessible name; group cards keyboard-operable; favicon; controls inherit the page font; UTF-8 BOM on the CSV export	Every listed acceptance criterion in §9 met; the previous-month button's rectangle is fully on screen at 375 px; the first month row is the newest	Viewport measurements at 375, 768 and 1280 px; DOM-order reads of the month rows; tab sweep reaching all cards	R3, and R4 for the retag before the plural fix	M

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R6	Make the money arithmetic, the message and the CSV testable for the first time	Create the app source tree, root <code>package.json</code> and <code>tsconfig</code> project references; port <code>App.utils</code> and <code>App.services.groups / csv</code> as pure typed functions with explicit parameters; encode the currency whitelist as a union type; rewrite the message generator to take template and overrides as parameters; add Vitest unit tests including the known-bad inputs	Every function named in QW-30 has a typed export and at least one test; <code>index.html</code> byte-unchanged; a grep over every new file finds no IBAN shape and no 8+ digit run	<code>npm run test:unit</code> green; <code>git status</code> shows <code>index.html</code> , <code>AGENTS.md</code> and <code>LICENSE</code> unmodified	R3	M
R7	Pin the only two outputs whose regression is silent	Approval tests for the substituted payment message and the serialized CSV bytes, using a neutral synthetic template; label every approved file contract or defect , naming the defect id where it applies	Approved files committed; a second run rewrites nothing; changing one fixture price by one unit fails exactly the affected test	Run twice, confirm <code>git status</code> clean; perturb one fixture and confirm the failure is the expected one	R6	S

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R8	Introduce the toolchain and a deployed React skeleton that cannot touch her data	Vite plus React plus TypeScript under the app root with <code>base: '/lesson-planner/'</code> ; the assemble script and <code>scripts/assert-storage-keys.sh</code> ; the artifact-deploy workflow with <code>ROOT_APP: legacy</code> ; prove one green run; then flip Settings → Pages to GitHub Actions; build the skeleton at <code>/lesson-planner/next/</code> reading and writing prefixed keys; port the storage module from R6	The live URL still serves the current page byte-identically after the flip; <code>/lesson-planner/next/</code> renders a JSON export of her data and one add-persist-reload cycle survives; its writes round-trip back through the legacy app; the key gate is wired	<code>curl -sI</code> the live URL before and after the flip and compare the etag; export from legacy, import into staging, mutate, export, import back into legacy	R6, D3	M
R9	Reach parity, with the defects fixed rather than reproduced, and accessibility done once	Work the RP-01 feature inventory as a committed checklist with a disposition per row; make the edit draft the single write target for the whole group-editing session; route money and message through the R6 modules; keyboard-operable calendar grid; real dialogs with focus trap and restore; palette pass to WCAG 2.2 AA; <code>role=status</code> announcements; accessible names; landmark; month-row ordering and the copy-this-month action; the fourth key and the sync-indicator surface	Every inventory row has a disposition — implemented, deliberately dropped with a reason, or deferred with a reason; every RP-01 defect fixed or deferred with a reason; the axe violation baseline only shrinks	The dual-project diff: the same spec set produces identical outcomes on both projects except the enumerated DEF rows, which fail on legacy and pass on React	R8	L

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R10	Move the live URL with a proven escape hatch	Preconditions checklist; set the storage-key prefix to empty; <code>ROOT_APP: react</code> ; retire the legacy Playwright project; local build, serve and full suite; hand-confirm the key gate; smoke test on a fresh profile; merge, watch both jobs, wait out the 10-minute cache; verify the <code>/legacy/</code> path serves the old page before telling her anything	The root URL serves the React build; all three keys present and <code>groupLessonPlannerData</code> parses; <code>/lesson-planner/legacy/</code> serves the old page; she has both URLs and one sentence of instruction, and has taken an export	DevTools on the live origin: three keys present and parsing. Rollback rehearsed on paper: one-line <code>ROOT_APP revert</code>	R9, plus a fresh off-laptop export	S
R11	Close the incident's root cause with zero end-user discipline	Firebase project in <code>europe-central2</code> ; Google provider only; <code>signInWithPopup</code> from a visible button; authorized-domain entry; the four-line security rule; <code>users/{uid}</code> whole-document debounced write-behind; boot-time fingerprint compare and push; the sync indicator driven by <code>hasPendingWrites</code> / <code>waitForPendingWrites</code> ; client-rotated snapshot documents with deterministic retention; read-only refusal on a newer <code>schemaVersion</code> ; sign-out confirmation	She signs in once and the indicator reaches <code>Backed up HH:MM</code> ; a wiped profile restores by sign-in alone; the indicator does not advance while offline; snapshot rotation leaves exactly the retained set; a second Google account cannot read the document	Procedure A rehearsed on a second device; offline test asserting the indicator stays behind; rules tested from a second account	R10, R0 step 5	M

Step	Goal	Tasks	Done when	Verified by	Depends on	Effort
R12	Spend the capability budget the storage layer unlocked	sentAt per group-month rendered in the row; per-group template override plus named variants; extra merge tokens with an unknown-token warning	The month row shows when it was last sent; a reminder variant survives a reload; <code>{{group}}</code> substitutes	Manual walkthrough of one month's billing with her	R11	M
R13	One app, one toolchain, nothing lying	Delete the <code>/legacy/</code> fallback after an agreed observation period; delete the legacy Playwright project and its DOM specs; delete pins now duplicated by unit tests; delete the dead CSS; bring <code>AGENTS.md</code> into line with the scripts that exist	No duplicated coverage; every documented command exists and passes; no fallback path remains	Fresh clone, run every documented command	R11, plus an agreed observation period with no rollback	S

6. Critical path and droppable work

The critical path has two independent halves, and only the second is a chain.

The first half is **R0 → R1 → R2**. It is on the critical path for one reason: it is the only work that reduces the probability of another total loss, and every day it is not done is a day of exposure. It does not depend on any decision, any toolchain, any hosting change or any answer from anyone. It should be finished before R3 starts, and it is deliberately sequenced so that the single most exposed thing — a real IBAN and tax identifier on the public internet — is dealt with on day one rather than at the end of a refactor.

The second half is **R3 → R6 → R8 → R9 → R10 → R11**. Each link is genuine. R3 freezes the file so line citations and commands stop being wrong. R6 produces the typed modules the skeleton's storage adapter is built from. R8 cannot start until D3 is answered, because the publishing-source flip is outside the code. R9 is the only L in the plan and the only step whose scope can silently grow. R10 is gated on R9's parity checklist having a disposition per row. R11 is gated on R10 because sync ships once, into the app that will still exist.

Off the critical path, and safely parallel or deferrable: R4 (the suite is a safety net, not a gate — branch publishing cannot be gated anyway), R5 (every item is independently shippable and none blocks anything), R7 (valuable, cheap, but the pins guard a regression rather than enable a step), R12 (pure capability work), R13 (cleanup). One ordering constraint holds between two off-path steps: R4's retag of the two specs that assert the current plural strings must land before R5's pluralisation fix, or the suite goes red for a reason that is not a regression. Nothing else in R4 and R5 is ordered relative to the other.

Droppable without compromising the outcome, stated as drops rather than deferrals.

Dropped	Why it can go
The 17-file, 412-line screenplay layer over six page objects	Each user action is currently expressed three times. The pattern earns its keep on a large team with a shared vocabulary, not for one maintainer with seven spec files
jsdom component tests with React Testing Library	C7. Two suites asserting the same behaviour disagree; the pure-function layer plus E2E covers it
Pixel visual regression	Snapshots are named per browser and platform, every icon is an OS-rendered emoji, and the most visually broken thing in the app has a 1.00:1 contrast ratio and is therefore invisible to a pixel diff by construction
Cross-browser matrix, Firefox and WebKit lanes	Triples runtime and flake surface for one user on one browser nobody has identified yet. Run Chromium; add her engine when it is known
Mutation testing, coverage thresholds, Codecov	A gate that gets lowered rather than met. Meaningful coverage here is "the money maths and the CSV parser have tests", which is checkable by reading a directory
Husky, lint-staged, commitlint, semantic-release, changesets, Renovate or Dependabot bump PRs	Ceremony for a solo maintainer who is also the only reviewer. Take GitHub's free security alerts, which are a setting and produce no noise
Storybook, Tailwind, a component library, a state library, a router, a monorepo tool, production source maps, bundle-size budgets	Each is standard advice at a scale this app will never reach. 224 lines of CSS, one screen, one user
A subscribable iCal feed, PDF invoices, payments, automated sending, portals, per-student records, attendance, an inbox, push notifications, a search box, a full RFC 5545 recurrence engine, a native app, an i18n framework, an analytics dashboard, an accessibility overlay widget	RP-08 §8's rejection list, adopted whole. Each needs a server, a second party, or a data model several times larger than the task
Client-side encryption of the free-text group name	It is theatre. The key must live in the browser (dies with the profile — the exact incident), be derived from a passphrase she can forget (trades a small confidentiality gain for a large availability loss), or sit beside the ciphertext. The honest substitute is not storing an identifying string at all
CRDTs, local-first sync engines, any Postgres, edge databases, self-hosted BaaS	One writer, about 14 KB, no server. The correct conflict policy is one line comparing two timestamps

Dropped	Why it can go
Firebase App Check, at first	With a correct <code>uid == uid</code> rule an attacker cannot reach her document. Trigger: daily Firestore usage deviates from what one teacher can generate
44 px touch targets, <code>.ics</code> download, <code>navigator.share()</code> , marking the Ukrainian block's language, retention automation	Deferred, not dropped. The first three need her device known; the last must never be automated in an app with this loss history

7. Consolidated TBD register

One question closes a cluster. Seven of the eight reports independently flagged the same top unknown: **which browser, device and operating system she uses.** It is not a nice-to-have detail, and it is not resolvable from any artifact — it needs one message. The answer resolves or re-scopes, at minimum: T1 below, and with it T2, T3, T4, T5, T6, T7 and T8. Ask it in the same message as §4 step 4.

#	Item	Why unresolved	Action to resolve
T1	Her browser, device and OS, on the old machine and the current one	Not in any artifact. Flagged independently by RP-02, RP-03, RP-04, RP-05, RP-06, RP-07 and RP-08	Ask her: Safari, Chrome, Edge or Firefox, on a Mac, a Windows laptop, an iPad or a phone. Or read one User-Agent from any support exchange
T2	Whether WebKit's seven-day script-storage sweep contributed to the original loss, and how often it will sign her out of Firebase	Depends on T1. The sweep clears exactly the storage Firebase Auth's persisted session lives in	T1. On Chromium and Firefox the session persists to explicit sign-out; on Safari expect a weekly sign-in, which costs one click and returns the data automatically
T3	Whether <code>signInWithPopup</code> works on her browser	Depends on T1, and it decides whether sign-in works at all rather than how often	T1, then confirm the popup flow on that browser before R11 ships
T4	Whether the default Vite build target reaches her browser	Depends on T1. A below-baseline browser fails with a blank page — the same symptom as a <code>base</code> misconfiguration, which will make it easy to misdiagnose	T1. If she is below the baseline, set <code>build.target</code> explicitly; do not reach for a legacy plugin
T5	Whether the phone findings affect her at all	Depends on T1. Changes "phone users are affected" to "she is or is not affected"	T1

#	Item	Why unresolved	Action to resolve
T6	Whether a Chromium persistent File System Access grant removes the per-backup gesture	Depends on T1 and on an untested combination of two documented facts [S22][S23]	T1, then test on Chrome 122+: store a handle, reload, call <code>queryPermission()</code> then <code>createWritable()</code> with no user gesture
T7	Which <code>localStorage</code> container path to search on her machine	Depends on T1; Apple does not document the Safari path and it moves between releases	T1, then locate it with <code>find</code> rather than typing a path from memory
T8	Whether <code>navigator.share()</code> is worth implementing, and which app she pastes into	Depends on T1 plus one more question	Ask which app she sends the message in, and evaluate 'share' in <code>navigator</code> on her device
T9	Real dataset scale — groups, months, dates: TBD	The original data was destroyed with the laptop	Ask her. Every scale figure in the programme comes from a constructed dataset and is labelled as such
T10	Whether she ever used Save CSV, and where those files went	Not in any artifact, and it is the most likely surviving copy	Ask her, then search every device for <code>lesson-planner-*.csv</code>
T11	What "the laptop was cleaned" meant, and whether the Mac is Apple silicon or T2	Not in any artifact, and it decides whether any recovery path exists at all	Ask. If it was Erase All Content and Settings on Apple silicon or a T2 Mac, the data is cryptographically inaccessible and recovery stops there
T12	Whether a Time Machine or other backup disk exists, and its oldest snapshot date: TBD	Not in any artifact	Ask, then <code>tmutil listbackups</code> before any new backup runs
T13	Conformance level of WCAG 2.2 SC 2.5.8: TBD	Neither RP-01 nor RP-08 states it, and this report did not fetch the specification	Read the conformance table at the WCAG 2.2 Recommendation [S24]. C6 does not depend on the answer: the target-size verdict is "passes at AA" either way
T14	Built bundle wire weight and request count after migration: TBD	Nothing has been built	Run the production build once and read the reported chunk sizes. Expect the app to be heavier and slower to first paint than the 13,610 gzipped bytes in one request it serves today — the migration's justification is maintainability, not performance, and it must not be sold as such

#	Item	Why unresolved	Action to resolve
T15	Failure mode of running a Pages workflow while the source is still branch-based	Not documented	One trial run, sequenced so the first Actions deployment publishes byte-identical content and the flip is content-neutral
T16	Whether Jekyll processing applies to an Actions-deployed artifact	Not stated on the fetched pages. Moot today, since the build emits <code>assets/</code> , not an underscore-prefixed directory	Publish one underscore-prefixed file and check whether it is served
T17	Whether Vite normalises a <code>base</code> value given without a trailing slash	Not stated on the config page	Build once with the slash omitted and inspect the emitted asset URLs. Until then, hardcode both slashes rather than wiring in the Pages action's <code>base_path</code> output, which has no trailing slash
T18	Version to pin for <code>@axe-core/playwright</code> , <code>actions/configure-pages</code> and <code>actions/deploy-pages</code> at implementation time: TBD	Version numbers go stale and two primary sources already disagree with each other on action majors	Read each package's registry entry and each action's own Releases API immediately before pinning. Each action's repository is authoritative for its own current major; the documentation prose lags
T19	Whether the Firebase and Cloud data-processing terms apply to no-cost Spark usage	Neither document addresses payment status	Read the Firebase and Google Cloud Terms of Service for whether the addendum is incorporated for no-charge services. Does not block: if the answer were no, it is an argument for the local-only fallback, which has no processor at all
T20	Who is named in the default payment template, and whether the maintainer is established in the EU	Not in any artifact; no attempt was made to identify the person	Ask him. D6 above
T21	Whether any current or future group name identifies a person	Her dataset is gone, so no artifact survives	Ask her what she called her groups. The non-identifying-label convention makes the answer moot going forward
T22	Retention period required by Polish tax and accounting rules: TBD	Deliberately not researched by any report, and it must not be guessed	Her accountant, or the Polish statutes. D9 above

#	Item	Why unresolved	Action to resolve
T23	Appeal status of the EU-US adequacy litigation, and the substance of EDPB Guidelines 07/2020	An appeal is reported only in secondary sources; the guidelines PDF did not parse	Search the Court's case database for an appeal against the first-instance judgment; fetch a text-extractable copy of the guidelines. Neither blocks anything: the data stays in Warsaw and the provider states it relies on standard contractual clauses where adequacy does not cover a transfer
T24	ePrivacy consent position for Firebase Auth session storage	Out of scope for every report; the Directive was not consulted	Read Art. 5(3) of the ePrivacy Directive as implemented in Polish law, plus any supervisory-authority guidance on strictly-necessary storage
T25	Actual suite runtime: TBD . Also whether Firestore TTL policies work on Spark	The suite was never executed against the committed file; the TTL page states no plan requirement, which is an absence rather than a confirmation	Run the suite after R3 and read the reporter duration. TTL is immaterial — the snapshot layer uses client-side rotation precisely so it does not depend on the answer
T26	Exact localStorage byte-versus-character basis of the measured ceiling: TBD	The specification source does not state which basis its figures use	Read the browser's DOM storage quota implementation. Nothing depends on it: the three-year dataset is about 70 KB against a measured ceiling three orders of magnitude larger

8. Project acceptance criteria

Three bodies of work, three separate definitions of done. Each condition is checkable by a command, a measurement or a yes/no observation — none is an adjective.

A. Data safety — the precondition for calling anything else finished.

1. The repository guard exits zero on `index.html` , and no IBAN shape, no run of eight or more digits, and no case-insensitive `IBAN` or `MF0` appears anywhere in the repository outside the permitted Ukrainian payment-purpose line.
2. A truncated value planted in `groupLessonPlannerData` leaves the month select with 12 options, the empty state rendered, `+ Add Group` working, the console error list empty, and the original unparseable string preserved under a `.corrupt.backup` key.
3. Export, clear, re-import restores all three keys byte-for-byte including the payment template, and the fingerprint is identical before and after.
4. Four consecutive destructive actions leave exactly three snapshot generations, and the undo restores all three keys — including an absent `paymentTemplate` staying absent rather than becoming an empty string.

5. An import into non-empty data shows a confirmation naming both sides' counts; cancelling changes nothing and the same file can be chosen again.
6. The last-backup indicator reads `Never backed up` with warning treatment on a fresh profile, and warns past 14 days.
7. Procedure B has been rehearsed end to end with the end user while nothing was broken, and the outcome was written down.

B. Migration — done when the diff is boolean, not when it looks finished.

8. The same spec set, run against the legacy and React projects, produces an identical pass-or-fail result for every test **except** the enumerated DEF rows, which fail on legacy and pass on React. This is a diff of two machine-readable reports, not a judgement.
9. Every row of the RP-01 feature inventory carries a committed disposition: implemented, deliberately dropped with a reason, or deferred with a reason.
10. Every RP-01 defect is either fixed with a test proving it, or deferred with a reason. No defect is silently carried.
11. No retained test locates an element by `#id`, `class`, `XPath`, or a `page.evaluate` reach-in into the app object.
12. Every test derives its dates from the fixed clock; no test reads the wall clock.
13. The three storage key names and the persisted shape are asserted by name and by parsed structure, in both directions, and a rollback to the legacy app reads what React wrote.
14. `scripts/assert-storage-keys.sh` passes on the cutover build, and the staging build's key prefix is non-empty.
15. The live URL serves the React build; `/lesson-planner/legacy/` serves the old page and has been opened successfully **before** the end user was told anything.
16. All three keys are present on her origin after cutover and `groupLessonPlannerData` parses. "The page loads" is not the criterion — a page that loads showing nothing is exactly what data loss looks like here.
17. `AGENTS.md` describes only commands that exist, and a fresh clone runs every one of them.

C. Storage replacement and improvement work.

18. A profile wiped after sign-in is restored by signing in alone, with no file and no developer involvement.
19. The sync indicator does **not** advance while the device is offline, proven by a test that asserts it stays behind. It advances only on server acknowledgement.
20. Closing the tab with a write pending, then reopening, results in the local state reaching the server — proven by a boot-time fingerprint comparison, not by trusting the SDK queue.
21. A second Google account cannot read or write the document, and no client can list the `users` collection.
22. Snapshot rotation leaves exactly the retained set — newest 14 daily plus first-of-month for 12 months — verified against a fixed document-id list before it ever runs on real data.
23. A client whose compiled schema version is lower than the stored one writes nothing at all: not to `localStorage`, not to Firestore, and it strips no field it does not recognise.

24. The month row shows when that month was last sent, and the accessibility violation baseline has only ever shrunk since it was first committed.
25. Under WCAG 2.2 Level AA: the calendar is operable by keyboard, dialogs have a role, an accessible name, a focus trap and focus restoration, no closed dialog is in the tab order, every form control has an accessible name, status changes are announced, and no text or non-text contrast pair measured in RP-08 §3 remains below its threshold.
26. The recurring monthly task is two clicks from the group card, and the current month is visible without scrolling when the group opens.

9. Consolidated quick-wins backlog

Standing warning, read before pasting any prompt below. Every prompt quotes line numbers against the **1473-line committed `index.html`** at commit `211eb6b`. QW-22 lands the diverged working copy, which is **1491 lines**, so from that moment every citation in this section, and in RP-01 through RP-08, is shifted by up to 18 lines. Two consequences. First, QW-22 is deliberately positioned **after** every other `index.html` quick win for exactly this reason. Second, R3 carries a tracked task to re-verify the citations in every unshipped prompt against the new file, with "no unshipped prompt cites a line number that has moved" as its done-when. If a prompt is run out of order, locate the target by symbol name — `App.services.storage.load`, `App.render.createMonthRow`, `App.handlers.updateDefaultPrice` — and not by line number.

The arithmetic, shown so it can be checked. The eight reports issued **40** quick wins ($6 + 5 + 5 + 4 + 5 + 4 + 5 + 6$). Six pairs describe the same change and are **merged**, reducing the count to **34**. **Nothing was dropped as invalidated** — that is a finding, not an omission: every quick win the programme produced survives reconciliation, and the six merges are all cases where two reports found the same defect from different angles. The merges are: the calendar-control wrap (RP-02 QW-1 with RP-08 QW-3); the favicon (RP-02 QW-3 with RP-04 QW-4); the CI workflow (RP-03 QW-1 with RP-04 QW-2); the `.gitignore` additions (RP-04 QW-3 with RP-06 QW-4); the numeric-input constraints (RP-01 QW-3 with RP-08 QW-2); and the group-name field (RP-06 QW-3 with RP-08 QW-6). **Twelve** prompts are **reissued** because a merge or a reconciled decision altered their scope, and each is marked `reissued` in the `From` column. The other **twenty-two** carry their originating report's text. In exactly three of those twenty-two a single Constraints line has been added where another quick win in this list creates an interaction the original report could not have known about — the group-name escaping and the group-card accessible name touch the same interpolation, the CSV byte-order-mark change invalidates an assertion the CSV export spec deliberately pinned, and the round-trip test must now expect the import confirmation dialog. Those three additions are visible in the prompt text and change no Task. No other carried prompt differs from its originating report.

How to read the table. `Rank` is impact divided by effort, with data-loss and exposure prevention weighted highest because the programme exists because of a loss. `Position` is the order to actually execute in, which respects dependencies and the line-number warning above. **Where Rank and Position disagree, follow Position.** `Independently shippable` is per item, not a blanket claim: six items genuinely are not, and each says what it needs.

Rank	Quick win	From	Effort / Impact	Independently shippable	Position
1	Raw storage dump procedure for every surviving device	RP-07 QW-1	XS / Critical	Yes — no code change, no deploy	Today, before anything else
2	Remove the published personal payment identifiers, and fix the template's plural wording	RP-01 QW-1, reissued	XS / Critical	Yes	R0, day 1. Blocks R11
3	Repository guard against personal financial identifiers	RP-06 QW-2	XS / High	Yes — correctly fails on today's file	R0, immediately after rank 2
4	Guard the storage layer and add a recovery path	RP-05 QW-1	S / Critical	Yes	R1
5	Escape the group name in the card sink	RP-06 QW-1	XS / High	Yes	R1
6	Constrain the year and price inputs, and clamp negatives	RP-01 QW-3 with RP-08 QW-2, merged and reissued	XS / High	Yes	R1
7	Sanitise the currency during CSV import	RP-07 QW-5	XS / High	Yes	R1
8	Versioned JSON backup envelope covering all three keys	RP-05 QW-2, reissued	S / Critical	Yes	R2
9	Snapshot all three keys before any destructive action, with an undo	RP-07 QW-2	S / Critical	Yes	R2
10	Confirm before an import replaces data, naming both sides	RP-05 QW-5, reissued	XS / High	Yes	R2
11	Data fingerprint utility for verifying imports and restores	RP-07 QW-4	XS / High	Yes	R2

Rank	Quick win	From	Effort / Impact	Independently shippable	Position
12	Additive schema-version key with a forward-compatibility refusal	RP-07 QW-3	S / High	Yes	R2
13	Last-backup age indicator with a nag	RP-05 QW-3	XS / High	Yes — it wires to Save CSV, which exists today, and gains Save Backup when rank 8 lands	R2, ideally after rank 8
14	Show the most recent month first in the group modal	RP-08 QW-1	XS / High	Yes	R5
15	Request persistent storage on load	RP-05 QW-4	XS / Medium	Yes	R2
16	Let the calendar controls wrap, and unclip the summary	RP-08 QW-3 with RP-02 QW-1, merged	XS / Medium	Yes	R5
17	Group-name field: no prefill, non-identifying placeholder	RP-08 QW-6 with RP-06 QW-3, merged and reissued	XS / Medium	Yes	R5
18	Refresh the group list after a schedule save	RP-01 QW-4	XS / Medium	Yes	R5
19	Name each copy button after the month it copies	RP-08 QW-5	XS / Medium	Yes	R5
20	Pluralise the two UI count strings	RP-01 QW-5, reissued	XS / Medium	Yes, but it turns two existing specs red — retag them first	R5, after R4's retag
21	Land the diverged working copy of <code>index.html</code>	RP-01 QW-2, reissued	S / High	Yes	R3 — after every other <code>index.html</code> quick win
22	Commit the toolchain files <code>AGENTS.md</code> already documents	RP-04 QW-1	S / High	Yes	R3
23	Make group cards operable from the keyboard	RP-08 QW-4	S / Medium	Yes	R5

Rank	Quick win	From	Effort / Impact	Independently shippable	Position
24	<code>.gitignore</code> : build output plus env and credential files	RP-04 QW-3 with RP-06 QW-4, merged and reissued	XS / Medium	Yes	R3
25	Copy the DOM-free domain logic into typed modules with unit tests	RP-02 QW-4, reissued	M / High (programme)	Yes — new files only	R6, brought forward
26	CI workflow: typecheck and run the suite, no deploy	RP-03 QW-1 with RP-04 QW-2, merged and reissued	S / High	No — needs rank 22	R4
27	Seed <code>localStorage</code> by overriding the <code>storageState</code> option	RP-03 QW-2, reissued	S / High	No — needs rank 22	R4
28	Freeze the clock before navigation and derive all test dates from it	RP-03 QW-3	S / High	No — needs rank 22	R4
29	Storage-contract spec for the three keys	RP-03 QW-5, reissued	S / High	No — needs rank 22	R4. The only test layer that survives cutover
30	Assert exported CSV content and a full round trip	RP-03 QW-4	S / Medium	No — needs rank 22	R4
31	Approval-pin the payment message and the CSV bytes	RP-02 QW-5	S / High (programme)	No — needs rank 25, or a local copy of the two functions	R7, brought forward
32	Favicon referenced at a subpath-relative href	RP-04 QW-4 with RP-02 QW-3, merged	XS / Low-Medium	Yes	R5
33	Buttons and form controls inherit the page font	RP-02 QW-2	XS / Low-Medium	Yes	R5
34	UTF-8 BOM on the CSV export	RP-01 QW-6	XS / Low-Medium	Yes	R5

Two ranking notes. Rank 14 sits above rank 15 despite both being XS because the month-ordering fix is the only change in the programme whose benefit **grows every month she keeps using the tool** — measured at 1,381 px of desktop and 2,627 px of phone scrolling to reach the one row she needs. And ranks 26 to 30 are ranked on their impact-to-effort ratio, which is high, but they sit at Position R4 because they are not shippable until the toolchain is committed; treating their rank as an instruction to run them first would fail.

PROMPT QW-1: Add a raw storage dump procedure that can be run today on any device
Context: Repo lesson-planner, single deployed file index.html at the repo root, served by GitHub Pages. The app's entire dataset is three localStorage keys: groupLessonPlannerData and groupLessonPlannerSettings, whose names are configured at index.html:377-378, and paymentTemplate, read and written as a bare string literal at index.html:1208 and index.html:1211. RP-01 verified that a corrupt value in groupLessonPlannerData makes App.init() throw, so any exporter that parses the data fails on exactly the data most in need of rescue. The app's existing CSV export at index.html:915-931 both parses and omits the template, so it cannot serve this purpose. Data recovery on any device that still holds data is time-sensitive and must not wait for a deploy.
Task: Create a new file docs/recovery/raw-dump.md containing a short, non-technical, numbered procedure for opening the browser developer console on the app's page and running a single self-contained JavaScript snippet, plus the snippet itself in a fenced javascript block. The snippet must read all three keys with localStorage.getItem and keep the results as raw strings without calling JSON.parse on any of them, wrap them in an object carrying a format identifier, an ISO timestamp and location.origin, and download the result as a .json file using a Blob and an object URL. The procedure must state that it should be run before installing updates, before importing anything, and on every device and browser that may still hold data. Do not modify index.html.
Constraints: Must not modify index.html or any other existing repository file. Must not parse, validate, repair or reformat the stored values in any way; a corrupt value must appear verbatim in the output. Must not delete or write any localStorage key. Must not use any library or any API beyond localStorage, Blob, URL.createObjectURL and a synthetic anchor click, so it runs in Chrome, Safari, Firefox and Edge. Must not include or invite any personal data in the document itself, and must warn that the dumped file contains the payment template and therefore personal payment details, so it must not be emailed or placed in a shared folder. Out of scope: importing the dump back, and any change to the CSV or JSON backup paths.
Acceptance criteria: The new file docs/recovery/raw-dump.md exists and contains exactly one fenced javascript block. Pasting that block into the console on a page with all three keys present downloads a .json file whose parsed content has a raw member with exactly the three key names, whose values are strings, and whose groupLessonPlannerData value is character-identical to the string returned by localStorage.getItem for that key. With groupLessonPlannerData set to the unparseable string [{"name":"Broken"}] the snippet still completes and the downloaded file contains that exact string. With paymentTemplate absent, the downloaded file records null for it rather than omitting the member. No localStorage key is added, changed or removed by running the snippet.
Verification: Serve index.html locally over http://127.0.0.1, seed one group, run the snippet from the console and open the downloaded file to compare the three values against localStorage.getItem output. Repeat with the truncated value planted in groupLessonPlannerData and confirm the download still succeeds. Repeat with paymentTemplate removed. After each run, assert Object.keys(localStorage) is unchanged.

PROMPT QW-2: Remove the hardcoded personal payment details and fix the template's plural wording
Context: Repo lesson-planner, single deployed file index.html at the repo root, served by GitHub Pages. App.config.defaultTemplate at index.html:382-400 is the fallback payment

message used whenever the localStorage key paymentTemplate is absent, which is the case for every new user. Lines 387–392 contain a real person's full name, bank IBAN, tax identification number and bank identifiers; line 400 contains that person's first name as a signature. All of it is publicly readable via view-source on the live site. Separately, RP-08 verified that the same template block hardcodes the wording that produces "we will have 1 lessons" for any single-lesson month, at index.html:384, so that grammatical defect reaches the message students read and not only two internal UI strings. Both edits touch the same string literal and are therefore one change, not two.

Task: Replace the payment-details block at index.html:387–392 and the signature at index.html:400 with neutral placeholder text that tells the user to enter their own details, keeping the surrounding English structure, the three template tokens {{month}}, {{lessons}} and {{total}}, and the Ukrainian payment-purpose line at index.html:395 intact. In the same change, reword the lesson-count sentence at index.html:384 so it is grammatical for a count of one as well as for larger counts, without introducing a fourth token and without changing the three existing tokens. Do not reproduce any of the removed values anywhere in the repo, including comments, docs and commit messages.

Constraints: Do not rename or restructure App.config.defaultTemplate. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not touch the getTemplate fallback logic at index.html:1207–1209, so a user who already saved a custom template keeps it untouched and is unaffected by the wording change. Do not change the two UI count strings at index.html:1089 and index.html:1172, which are a separate prompt. No new dependencies, no build step. Out of scope: purging the values from git history, which is a separate decision the developer must make and which must not delay telling the named person.

Acceptance criteria: All four of these greps over index.html return zero matches, run with grep -E: the IBAN shape of two uppercase letters followed by six or more digits; any digit run of eight or more; any Cyrillic character on lines 382 to 394 or line 400 of the template block, tested with a Cyrillic character class using grep -P; and the fixed strings IBAN and MF0 case-insensitively. The one intentional Cyrillic line, the payment-purpose text at index.html:395, is the sole permitted exception and must remain byte-identical. The default template still contains {{month}}, {{lessons}} and {{total}} exactly once each. With localStorage empty, creating a group with exactly one lesson and clicking Copy Payment Message yields a message that reads grammatically for a single lesson, has all three tokens substituted, and contains no remaining brace characters. Repeating with four lessons also reads grammatically. A user with an existing paymentTemplate value still sees that value unchanged in the Edit Template modal.

Verification: Run the four greps above and confirm each returns nothing. Then serve the file locally, clear localStorage, add a group, select exactly one date, click Done, click Copy Payment Message, and read the review textarea. Repeat with four dates. Finally set paymentTemplate to a custom string, reload, open Edit Template, and confirm the custom string is shown unchanged.

PROMPT QW-3: Add a repository guard that fails when personal financial identifiers appear in index.html

Context: Repo lesson-planner. The default payment template at index.html:382–400 contained a real person's full name, bank IBAN, tax identification number and bank identifiers at lines 387–392, plus a personal first name at line 400, all publicly readable through view-source on the deployed GitHub Pages site and present in git history. A separate prompt removes them from HEAD. Nothing currently prevents an equivalent value being pasted back in during a future edit, and there is no CI in the repository: no .github directory exists and every npm script named in AGENTS.md is aspirational because no root package.json is committed.

Task: Create one new executable shell script at the repository root named check-no-personal-data.sh that scans index.html and exits non-zero, printing the offending line numbers, when any of the following is found: a string matching the IBAN shape of two uppercase letters followed by six or more digits; any run of eight or more consecutive digits; or the case-insensitive fixed strings IBAN and MF0. The script must allow exactly one exception, the

Ukrainian payment-purpose line, identified by content rather than by line number so the check does not break when lines move. The script must print a short explanation of what to do when it fails, and exit zero with a one-line success message otherwise.

Constraints: Do not modify index.html. Do not create a .github directory or any CI workflow; wiring the script into CI is explicitly out of scope and should be mentioned only in the script's own comment header. Do not add a package.json or any npm script. Do not modify AGENTS.md. Use only POSIX shell plus grep available on macOS and Linux; do not depend on GNU-only grep flags, and where a Perl-style character class is needed, detect its availability and fall back rather than failing. Do not embed any real name, IBAN, tax identifier or bank identifier in the script, in its comments, or in its output; the script must match shapes and generic keywords only. Do not scan git history.

Acceptance criteria: Run against the index.html committed today, the script exits non-zero and reports the offending lines inside the 382 to 400 range; this failure is the expected and correct result and is the proof the guard works, because those values have not yet been removed. Run against a copy of index.html with the payment-details block and the signature line replaced by neutral placeholder text, it exits 0 and prints the success line. Run against a copy into which a string of the shape two uppercase letters followed by twenty digits has been inserted, it exits non-zero and prints that line's number. Run against a copy containing the word iban in lower case, it exits non-zero. The permitted Ukrainian payment-purpose line alone never causes a failure. The script's own source contains no digit run of eight or more characters. The script accepts an optional file path argument, defaults to index.html relative to the script's own directory, and runs correctly from any working directory.

Verification: Run the script from the repository root and again from a different directory and confirm both report the same non-zero result and the same line numbers against the current file. Copy index.html to a scratch path, replace the payment-details lines and the signature line with neutral placeholder text, run the script against that copy and confirm exit 0 and the success line. Append to that cleaned copy a line containing two uppercase letters followed by twenty digits and confirm a non-zero exit naming that line. Repeat with a line containing the lower-case word iban. Finally delete the Ukrainian payment-purpose line from the cleaned copy and confirm the script still exits 0, proving the exception is not load-bearing for success.

PROMPT QW-4: Guard the storage layer and add a recovery path

Context: Repo lesson-planner, single deployed file index.html at the repo root, served by GitHub Pages. App.services.storage at index.html:1187-1213 has no error handling. load() calls JSON.parse on the raw localStorage value at index.html:1192 and index.html:1195 and assigns the result straight to App.state.groups. RP-01 runtime-verified that planting a truncated value in groupLessonPlannerData throws an uncaught SyntaxError that aborts App.init() after cacheElements() but before the month dropdown is populated at index.html:413-415, before bindEvents() at 420 and before render.groups() at 421, leaving a page with no groups, no empty state, zero month options and inert buttons. save() at index.html:1199-1202 is equally unguarded against QuotaExceededError and against the SecurityError some browsers raise when storage is blocked.

Task: Wrap each JSON.parse in load() in its own try/catch so a failure in either key cannot prevent the other from loading and cannot prevent init() from completing. On a parse failure, copy the unparseable raw string to a new localStorage key by appending the suffix .corrupt.backup to the original key name, leave App.state at its default, and surface a single non-blocking message telling the user the saved data could not be read and that a copy has been kept. Wrap the two setItem calls in save() in a try/catch that reports a failure to the user without discarding App.state. Add a defensive shape check in load() so App.state.groups is assigned only when the parsed value is an array, and App.state.defaultCurrency only when the parsed settings value is a non-null object.

Constraints: Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not change the call order inside App.init() at index.html:409-422. Do not introduce a

schema version field in this prompt. No new dependencies, no build step. Out of scope: cloud sync, and changing the CSV import or export.

Acceptance criteria: With `groupLessonPlannerData` set to the string `[{"name":"Broken"}` and the page reloaded, the month select has 12 options, the group list shows the exact empty-state text `No groups yet`. Click `' + Add Group'` to get started!, clicking `+ Add Group` opens the Add Group modal, and no uncaught error appears in the console. A key named `groupLessonPlannerData.corrupt.backup` exists and contains the original unparseable string. With `groupLessonPlannerSettings` set to garbage and `groupLessonPlannerData` valid, the groups still render. With both keys valid, behaviour and persisted bytes are unchanged from before this prompt.

Verification: Serve `index.html` locally over `http://127.0.0.1`, then for each of the three cases above set the `localStorage` value from the console, reload, and assert the month option count, the group list text content, the presence of the backup key, and an empty console error list. Finally add a group, reload, and confirm the persisted JSON is byte-identical in shape to the RP-01 reference sample.

PROMPT QW-5: Escape the group name before interpolating it into the group card

Context: Repo `lesson-planner`, single deployed file `index.html` at the repo root, served by GitHub Pages. `App.render.groups` builds each card in a helper at `index.html:1042-1053`, which assigns `card.innerHTML` from a template literal that interpolates the group name unescaped at `index.html:1047`. RP-01 runtime-verified that a group name supplied through CSV import containing an `img` tag with an `onerror` attribute parses into a real DOM element and executes, that the payload persists to the `groupLessonPlannerData` `localStorage` key, and that it re-fires on every page load. There is no escaping helper anywhere in the file today; `App.utils` sits at `index.html:1365` onwards. This is also the prerequisite that stops a future signed-in sync session turning an injected script into a data-exfiltration path.

Task: Add a small pure function to `App.utils` that returns an HTML-escaped copy of a string, escaping at minimum the ampersand, less-than, greater-than, double-quote and single-quote characters, escaping the ampersand first. Apply it to the group-name interpolation at `index.html:1047` so the card renders the name as literal text.

Constraints: Do not change the rendered DOM structure, class names, element order or the surrounding text of the card, including the string that produces the planned-lessons count at `index.html:1049`. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape; escaping is applied at render time only and stored values must remain byte-identical. Do not alter the sanitisation or validation of CSV import in this change. Do not switch the card to `createElement` or `textContent`; keep the `innerHTML` template so the change stays a one-line diff plus a helper. No new dependencies and no build step.

Acceptance criteria: A group whose stored name is a bold-tag wrapper around the single letter `x` renders on the card with every angle bracket and slash visible as literal text, and produces no `B` element inside the card. A group whose stored name contains an `img` tag with an `onerror` attribute produces no `IMG` element in the document and fires no script. A group named `A & B` still displays as `A & B`. The stored value of `groupLessonPlannerData` is unchanged by rendering. Opening a group, editing its name and saving still round-trips the exact typed characters, including angle brackets, back into the name input.

Verification: Serve `index.html` locally over `http://127.0.0.1`, seed `groupLessonPlannerData` with one group whose name is an `img` tag carrying an `onerror` attribute that sets a global marker variable, reload, then assert in the console that the marker is undefined, that the count of `img` elements inside `.group-card` is `0`, and that the card text contains the literal tag characters. Repeat with the name `A & B` and assert the card text is exactly `A & B`. Finally read `groupLessonPlannerData` back and assert the name string is unchanged.

PROMPT QW-6: Constrain the year and price inputs so the app cannot create data that breaks itself

Context: Repo `lesson-planner`, `index.html`. Two separate input classes are unvalidated and both

produce user-visible damage. First, the year input at index.html:320 is type=number with no min, max or step; updateCalendarYear at index.html:823-828 does Number(value) and stores it unvalidated in App.state.calYear; App.utils.iso at index.html:1394-1396 pads only the month and day, so a year of 5 produces date keys like 5-08-10, and App.utils.toMonthKey at index.html:1398-1401 is a naive slice(0,7) that then yields a month key of 5-08-10 rather than a year and month. Verified consequence: such data persists, and re-importing the app's own CSV export throws Invalid month format from App.utils.normalizeMonthKey at index.html:1457-1466, aborting the entire restore; a blank input yields year 0 and negative years are accepted. Second, the default price input at index.html:287 and the bulk price input at index.html:332 are type=number with no min and no step, and updateDefaultPrice at index.html:690-705, handleSelectedDatesPriceChange at index.html:858-881 and the per-month handler at index.html:1110-1115 all accept the parsed value with no range check. Verified consequence: a price of -50 with one lesson produces an outgoing payment message whose fee line reads a negative currency amount, which is the message she copies and sends to students. These are one class of change to one file and are done together.

Task: Add min="1970", max="2100" and step="1" to the year input at index.html:320, and in updateCalendarYear clamp the parsed number into that range, falling back to the current calendar year when the value is blank or not finite, before calling setCalendarState. Add min="0" and step="0.01" to the price inputs at index.html:287 and index.html:332, and to the per-month price input created at index.html:1094, and clamp the parsed value to a minimum of 0 in all three handlers that read those inputs so a negative typed value is treated as 0 rather than stored.

Constraints: Do not change App.utils.iso, toMonthKey or normalizeMonthKey. Do not change App.utils.formatCurrency. Do not change the existing logical-or fallback semantics at index.html:662 and index.html:678 for blank or non-numeric price input, which must continue to yield 0. Do not add a maximum price. Do not change which months the bulk price handler writes to, which is a separate open product question. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape; this prompt prevents new corruption and deliberately does not migrate existing corrupt values. Keep the previous and next month buttons able to cross a year boundary within the allowed range. No new dependencies.

Acceptance criteria: Typing 5 into the year input and committing leaves the calendar on a year within 1970 to 2100 and never produces a date key whose year part is shorter than four characters. A blank or negative year does not set App.state.calYear to 0 or a negative number. Selecting any date and pressing Done produces only date keys matching four digits, hyphen, two digits, hyphen, two digits. Navigation across December to January still works. Typing -50 into the default price input and committing leaves the group price at 0 and the display shows a zero amount, not a negative one. Typing -50 into the bulk price input leaves every affected month override price at 0. Typing -50 into a per-month price input leaves that month's price and total at 0. A valid price of 200 still round-trips unchanged and a decimal price of 12.50 is still accepted. A generated payment message never contains a minus sign in the fee amount.

Verification: Serve locally, open a group, open the schedule editor, type 5 then Enter in the year input and confirm the rendered year is clamped; repeat with an empty value and with -3. Select a date, press Done, read groupLessonPlannerData and assert every entry in dates and every key of monthlyOverrides has a four-digit year. Then type -50 into Default Price, blur, and read the stored group price and the on-screen value; repeat with the bulk price input while dates are selected and read every price in App.state.tempMonthlyOverrides; repeat with a per-month price input and read the row total. Finally generate a payment message for a month with one lesson and assert the fee amount contains no minus sign.

PROMPT QW-7: Sanitise the currency during CSV import so a bad value cannot brick a group

Context: Repo lesson-planner, index.html. App.services.csv.deserialize at index.html:1283-1360 accepts any non-empty string as a group's currency at index.html:1317.

App.utils.formatCurrency uses Intl.NumberFormat, which accepts any three ASCII letters but throws RangeError otherwise. RP-01 runtime-verified the consequence: importing a currency of

US Dollar makes the group permanently unopenable, because the throw happens inside `render.monthlyOverrides()` which runs before the modal is shown, so `openGroupModal` aborts and there is no UI route to open, fix or delete that group; only Clear All Data or another import escapes. The currency select at `index.html:291–294` offers only UAH and PLN, and the app's fallback currency elsewhere is UAH.

Task: In `deserialize`, normalise each row's currency value before it reaches the group object: trim it, uppercase it, and accept it only when it matches exactly three ASCII letters. When it does not match, fall back to the existing default currency for that import and record the substitution. After the import completes, if any substitution was made, show a single non-blocking message naming how many groups had their currency replaced and what value was used, so the user is told rather than surprised later.

Constraints: Do not change the CSV column names, order or quoting, and do not change any existing error message, including the Unable to load CSV prefix used at `index.html:951`. Do not reject the whole import because of a bad currency; a single bad cell must never abort a restore. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not add validation to the currency select, and do not attempt to repair currencies already stored in `localStorage`; this prompt guards the import path only. Do not use `alert` for the substitution notice. No new dependencies, no build step. Out of scope: whether a three-letter but meaningless code such as XYZ should be rejected, which remains accepted because `Intl` formats it without error.

Acceptance criteria: Importing a CSV whose Currency cell is US Dollar produces a group that opens normally, with a currency of exactly three uppercase letters, and a visible notice reporting one substitution. Importing a CSV whose Currency cell is `pln` produces a group with currency PLN. Importing a CSV whose Currency cell is empty produces a group whose currency is the import default rather than an empty string. Importing a well-formed CSV exported by the app produces byte-identical stored data to before this change. A CSV that previously threw for a missing column or an invalid month still throws the same message and still leaves existing data untouched.

Verification: Serve locally, create a group, export a CSV, then hand-edit copies whose Currency cell contains US Dollar, `pln` and an empty value. Import each and assert the group card opens, that the stored currency matches three uppercase letters, and that the notice appears only for the substitution cases. Re-import the unmodified export and assert `groupLessonPlannerData` is byte-identical to the value captured before the change. Finally import a CSV with a missing Name column and assert the original error message and that the existing data is unchanged.

PROMPT QW-8: Add a versioned JSON backup envelope that covers all three storage keys

Context: Repo `lesson-planner`, `index.html`. The only existing backup path is CSV: `saveToCsv` at `index.html:915–931` and `loadFromCsv` at `index.html:935–963`. RP-01 verified three defects that make it unfit as a backup – it omits the `paymentTemplate` key entirely, a date key with a malformed year makes re-import throw Invalid month format and abort the whole restore, and a balanced stray quote is silently accepted and replaces all existing groups with garbage. The app's three `localStorage` keys are `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, the last being a raw string rather than JSON. RP-07 defines a single canonical envelope shape that serves both the local file and, later, the cloud document, so this prompt implements that shape rather than a narrower one, to avoid a format migration inside the same programme.

Task: Add a Save Backup button and a Load Backup button to the toolbar at `index.html:229–243`, alongside the existing Save CSV and Load CSV controls, plus a hidden file input for the backup file mirroring the existing hidden CSV input at `index.html:239`. Save Backup downloads a single `.json` file whose top-level members are: `format`, holding the fixed string `group-lesson-planner-backup`; `schemaVersion`, holding the number 1; `exportedAt`, holding an ISO 8601 timestamp; `source`, an object holding origin taken from `location.origin`; and `data`, an object whose three members are named exactly `groupLessonPlannerData` holding the parsed array, `groupLessonPlannerSettings` holding the parsed settings object, and `paymentTemplate` holding

the raw string from that key or null when the key is absent. Load Backup reads a chosen .json file, validates that format equals the fixed string, that schemaVersion equals 1, and that data.groupLessonPlannerData is an array, and only then writes all three localStorage keys and re-renders. On any validation failure it must change nothing and report what was wrong. Name the downloaded file with a lesson-planner-backup prefix followed by a timestamp.

Constraints: Do not remove, rename or change the behaviour of the existing CSV export or import. Do not change the three localStorage key names or the persisted data shape of any key; the backup file is a container around the existing shapes, not a new format for them, and the member names inside data must be the literal key names so the mapping is auditable by eye. Do not add the counts or fingerprint members in this prompt; they are additive and are added when the fingerprint utility lands. Reuse the existing Blob and object-URL download approach from index.html:915–931 rather than introducing a library. No new dependencies, no build step. Out of scope: migrating or repairing existing malformed date keys, and any cloud upload.

Acceptance criteria: With two groups, a non-default currency and a customised template present, clicking Save Backup downloads a file whose parsed contents contain format equal to group-lesson-planner-backup, schemaVersion equal to 1, a parseable exportedAt, a data object whose groupLessonPlannerData array has length 2, a settings object, and the exact template string. Clearing all three keys and then loading that same file restores all three so that the group list, the currency formatting and the Edit Template contents all match the pre-export state exactly. When paymentTemplate was absent before export, the file records null and loading it leaves the key absent rather than setting it to an empty string. Loading a file whose schemaVersion is 2 changes no localStorage key and reports an error. Loading a file whose format string differs changes no key and reports an error. Loading a file whose data.groupLessonPlannerData is an object rather than an array changes no key and reports an error. Loading a file that is not valid JSON changes no key and reports an error. The CSV export and import continue to behave exactly as before.

Verification: Serve locally, build the two-group fixture, click Save Backup and inspect the downloaded JSON. Clear localStorage, reload, click Load Backup with that file, and assert the three keys byte-for-byte against values captured before the export. Repeat with paymentTemplate absent before export. Then hand-edit copies of the file for each of the four rejection cases and confirm after each attempt that Object.keys(localStorage) and every stored value are unchanged.

PROMPT QW-9: Snapshot all three storage keys before any destructive action, and add an undo Context: Repo lesson-planner, index.html. Two code paths destroy all existing data.

loadFromCsv at index.html:935–963 assigns App.state.groups from the parsed file and persists at index.html:948 with no snapshot and no undo; RP-01 runtime-verified that this silently replaced two real groups, and separately that a CSV containing a balanced stray quote replaced them with a single garbage group. clearAllData at index.html:964–975 confirms and then calls App.services.storage.clear() at index.html:1203–1206, which removes two keys and leaves paymentTemplate behind. There is no way back from either. The storage service is App.services.storage at index.html:1187–1213.

Task: Add a snapshot function to App.services.storage that copies the current raw string values of all three keys groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate into keys named by appending .snapshot.<epochMilliseconds> to each original key name, recording absent keys as absent rather than as empty strings. Call it immediately before the state assignment in loadFromCsv and immediately before storage.clear() in clearAllData. Add a matching restore function that writes a chosen snapshot generation back over the three keys and re-renders, and expose it through a toolbar control that is visible only when at least one snapshot generation exists, labelled to say it undoes the last replacement. Keep at most the three most recent generations, deleting older ones in the same pass.

Constraints: Do not change the three localStorage key names or the persisted data shape of any of them; snapshot keys are additive and no other code may read them. Do not change the parsing, validation or error messages of the CSV importer, and do not change the Clear All

Data confirmation text. Do not add snapshot keys to `storage.clear()`. Snapshots must be taken from raw `getItem` values without parsing, so a corrupt value is preserved verbatim. Use the confirm mechanism already used by Clear All Data rather than a custom modal. No new dependencies, no build step. Out of scope: the JSON backup file format, and cloud sync. Acceptance criteria: With two groups, a non-default currency and a customised template present, importing a CSV creates snapshot keys for all three original keys sharing the same epoch suffix, and the undo control then restores all three to their pre-import values exactly, including the template. Clear All Data also creates a snapshot generation, and undo after it restores the two cleared keys. Performing four destructive actions leaves exactly three snapshot generations. When `paymentTemplate` is absent before the action, restoring that generation leaves it absent rather than setting it to an empty string. When no snapshot exists the undo control is not shown. Importing a CSV that throws leaves the pre-existing data intact and does not leave a partial snapshot generation behind. Verification: Serve locally, build the two-group fixture with a custom template, export a CSV, import a different CSV, then read all snapshot keys from the console and assert they match values captured before the import. Click undo and assert the three live keys are byte-identical to the captured values and the group list and Edit Template contents match. Repeat for Clear All Data. Run four destructive actions and assert the count of keys whose name contains `.snapshot.` is exactly nine, being three generations of three keys.

PROMPT QW-10: Confirm before a file import replaces existing data, naming both sides
Context: Repo lesson-planner, `index.html`. `loadFromCsv` at `index.html:935-963` replaces `App.state.groups` wholesale and persists at `index.html:948` with no confirmation of any kind. RP-01 runtime-verified that importing a file silently replaced two existing groups with the imported content and showed zero dialogs, and separately that a CSV containing a balanced stray quote replaced both real groups with a single garbage group, again with no dialog. By contrast Clear All Data at `index.html:964-975` does ask, using the text Clear all groups and schedules? This cannot be undone. Import is the app's only restore path, so an accidental import is a data-loss event on the very control the user reaches for during a recovery. RP-07 establishes that a bare are-you-sure is not enough: the dialog must state both sides, because the incoming side can itself be garbage.
Task: Before any imported file is applied, and only when `App.state.groups` is non-empty, show a confirm dialog that names both sides – the number of groups and the total number of lessons currently on this device, and the number of groups and total lessons in the file about to be applied – and requires explicit approval. On cancellation, apply nothing, leave all `localStorage` keys untouched, and reset the file input so the same file can be chosen again afterwards. Apply the same guard to every file-import entry point present in `index.html`, which is the CSV import and, if that prompt has already landed, the JSON backup import. Constraints: The incoming counts must be computed from the already-parsed file content, after the existing parser has validated it and before anything is written, so a file that would throw still throws its existing error and never reaches the dialog. Do not change the parsing, validation or error messages of the existing importers, and keep the existing alert wording Unable to load CSV: followed by the underlying message intact. Do not add a confirmation when there are zero existing groups, since nothing can be lost. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Use the same confirm mechanism already used by Clear All Data rather than introducing a custom modal. No new dependencies, no build step. Acceptance criteria: With two groups holding eighteen lessons between them, choosing an import file containing four groups and ninety-six lessons shows a confirm dialog whose text contains the numbers 2, 18, 4 and 96. Cancelling leaves the group list, the month rows and all `localStorage` keys exactly as they were, and choosing the same file again still shows the dialog. Accepting performs the import exactly as before this change. With zero existing groups, choosing an import file shows no dialog and imports directly. A malformed file that previously threw still produces the same error message, shows no confirm dialog, and leaves existing data untouched.

Verification: Serve locally, create two groups with a known total lesson count, export a backup or CSV from a different fixture with a known different count, then import it and cancel; assert the two original group names and every localStorage value are unchanged, and that the dialog text contained all four numbers. Re-import and accept and assert the import applied. Repeat the cancel case twice in a row with the same file to prove the input was reset. Then clear all data and confirm an import proceeds with no dialog. Finally import a CSV with a missing Name column and confirm no dialog appears and the original error is shown.

PROMPT QW-11: Expose a data fingerprint utility for verifying imports, restores and migrations

Context: Repo lesson-planner, index.html. App.utils at index.html:1365 already holds pure helpers such as iso at index.html:1394-1396 and toMonthKey at index.html:1398-1401. There is no way to compare the dataset before and after a destructive operation, so an import, a restore or a future migration can only be checked by eye. RP-01 also verified a latent invariant problem: every date is stored twice, once in group.dates and once in monthlyOverrides[m].dates, synchronised only in saveDateChanges at index.html:745-772, so the two totals can diverge silently. JSON.stringify preserves insertion order, so a naive checksum of two semantically identical datasets can differ.

Task: Add a pure function App.utils.dataFingerprint() that reads the three localStorage keys, parses the two JSON ones defensively so a corrupt value yields a reported problem rather than a throw, and returns an object with the group count, the total of group.dates lengths, the total of monthlyOverrides dates lengths, a boolean stating whether those two totals are equal, the number of distinct month keys, the character length of the payment template or null when the key is absent, an array of problem strings listing every date key not matching four digits hyphen two digits hyphen two digits, every month key not matching four digits hyphen two digits, and every group whose currency is not exactly three ASCII letters, and a short hexadecimal checksum computed over a canonical form in which object keys are sorted and array members are sorted so the result is independent of insertion order.

Constraints: Must be read-only: it must not write, remove or reorder any localStorage key, and must not mutate App.state. Do not change the three localStorage key names or the persisted data shape. Do not write the canonical form back to storage anywhere. Do not surface it in the user interface in this prompt; it is a developer and verification tool reachable from the console. Do not add a dependency and do not use any hashing library; an inline non-cryptographic checksum is sufficient and must be deterministic across browsers. No build step. Out of scope: repairing any problem it reports.

Acceptance criteria: With a dataset of two groups, six month keys and forty-one dates present in both places, the returned object reports 2, 41, 41, true and 6, an empty problems array, and a checksum string. Reloading the page and calling it again returns an identical checksum. Reordering the members of one group's dates array in storage and reloading returns the same checksum. Adding one date changes the checksum. Planting a date key of 5-08-10 adds exactly one problem entry naming that value, and planting a currency of US Dollar adds exactly one problem entry naming that group. With groupLessonPlannerData set to an unparseable string the function returns a problems array describing it and does not throw. Calling it never changes Object.keys(localStorage) or any stored value.

Verification: Serve locally over http://127.0.0.1, build the fixture, and call App.utils.dataFingerprint() from the console, asserting each field. Capture all localStorage values before and after the call and assert they are identical. Reorder a dates array from the console, reload, and assert the checksum is unchanged. Then add a date and assert it changed. Finally plant the malformed date, the malformed currency and the unparseable value in turn and assert the problems array in each case.

PROMPT QW-12: Add an additive schema-version key and refuse to write when the stored version is newer

Context: Repo lesson-planner, index.html. RP-01 verified that none of the three storage keys carries a schema version, so no future migration has anything to branch on and no client can

tell whether stored data was written by an older or a newer build.

`App.services.storage.load()` at `index.html:1188–1198` assigns parsed values straight into `App.state`, and `save()` at `index.html:1199–1202` writes both config keys unconditionally. A durable-storage plan for this app writes the whole state as one document, so a client that does not understand a newer shape will silently delete the fields it does not know about the first time it saves.

Task: Introduce a module-level constant holding the current schema version as the integer 1, and a new additive `localStorage` key named `groupLessonPlannerSchemaVersion` holding that number as a decimal string. In `load()`, read the key, treat an absent or unparseable value as version 0, and treat 0 and 1 as loadable. If the stored value parses to an integer greater than the constant, do not assign anything into `App.state`, set a new `App.state` flag named `readOnlyReason` to a short string naming the version found and the version expected, and render a non-blocking message telling the user the data was saved by a newer version and to reload the page. In `save()`, return without writing anything when `readOnlyReason` is set. When loading succeeds and the key was absent, write the constant to the key on the next successful save rather than during load.

Constraints: Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape of any of them. Do not write the version key during `load()`; load must remain free of side effects on storage. Do not implement any migration function in this prompt; version 0 and version 1 describe the same shape. Do not block rendering and do not use `alert` or `confirm` for the newer-version case. No new dependencies, no build step. Out of scope: the Firestore document, the JSON backup envelope, and any repair of malformed data.

Acceptance criteria: On a profile with no version key, the app loads normally, and after any action that calls `save()` the key `groupLessonPlannerSchemaVersion` holds exactly the string 1 while the other three keys are byte-identical in shape to before the change. With the key set to 1, behaviour is identical to today. With the key set to 2, the group list does not render stored groups, a visible message mentions a newer version, `App.state.readOnlyReason` is a non-empty string, and after attempting an edit the three data keys are byte-identical to their pre-attempt values. With the key set to the string `abc`, the app loads normally and treats it as version 0. No uncaught error appears in the console in any of these cases.

Verification: Serve locally over `http://127.0.0.1`. For each of the four key states above, set the value from the console, reload, and assert the rendered group count, the presence or absence of the message, the value of `App.state.readOnlyReason`, and an empty console error list. In the version 2 case, capture all three data keys before and after clicking Add Group and Save and assert they are unchanged. In the absent case, add a group and assert the version key now reads 1.

PROMPT QW-13: Show how long ago the last backup was taken and nag when it is stale

Context: Repo `lesson-planner`, `index.html`. The app gives the user no signal about whether an off-device copy of her data exists. RP-01 records that all stored data was previously lost when the user's laptop was cleaned, and that the app's only persistence is three `localStorage` keys. There is currently no record anywhere of when a backup was last taken. This prompt depends on a backup action existing; it must therefore wire itself to whichever download action is present in `index.html` at the time it is implemented, which is Save CSV at `index.html:915–931` today and additionally Save Backup if the JSON backup prompt has already landed.

Task: On every successful backup download, write the current time as an ISO 8601 string to a new `localStorage` key named `lastBackupAt`. On page load, render a short status line in the toolbar area at `index.html:229–243` reading either Never backed up when the key is absent, or Last backup: today, or Last backup: N days ago. When the key is absent or the age exceeds 14 days, give that status line a visually distinct warning treatment using the existing danger colour already present in the stylesheet.

Constraints: Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape of any of them; `lastBackupAt` is an additive key that no other code reads. Do not add the new key

to the `clear()` function at `index.html:1203-1206` in this prompt. Do not block, modal or interrupt the user; this is a passive indicator only. Ensure the status line is announced to assistive technology by giving it an appropriate live-region attribute. No new dependencies, no build step.

Acceptance criteria: On a profile with no `lastBackupAt` key, the status line reads exactly Never backed up and carries the warning treatment. After clicking a backup download, the key holds a parseable ISO 8601 timestamp and the line reads exactly Last backup: today without the warning treatment. Setting the key to a timestamp 3 days in the past yields Last backup: 3 days ago without the warning treatment. Setting it 15 days in the past yields Last backup: 15 days ago with the warning treatment. No dialog or alert is shown at any point. The three existing keys are untouched by all of the above.

Verification: Serve locally, and for each of the four states above set or clear `lastBackupAt` from the console, reload, and assert the rendered text content of the status element and the presence or absence of its warning class. Trigger a real backup download once and assert the key was written and is parseable by Date. Confirm `Object.keys(localStorage)` gains only `lastBackupAt`.

PROMPT QW-14: Show the most recent month first in the group modal

Context: Repo lesson-planner, single deployed file `index.html` at the repo root, served by GitHub Pages. `App.render.monthlyOverrides` at `index.html:1055-1078` builds the month rows inside the group modal. Line 1067 sorts the YYYY-MM keys as strings in ascending order, so the oldest month renders first and the current month renders last. Measured on a seeded dataset of one group with 36 consecutive months of lessons: the current month is row 36 of 36, the modal content is 2282px tall in an 850px box at 1280x900 and 3453px in a 771px box at 375x812, and reaching the current month requires 1381px of scrolling on the desktop viewport and 2627px on the phone viewport. The current month is not visible when the modal opens.

Task: Reverse the ordering at `index.html:1067` so month rows render newest-first, by replacing the bare `sort()` with a comparator that sorts the YYYY-MM keys in descending order. Change nothing else about how the rows are built.

Constraints: Do not change the row markup produced by `App.render.createMonthRow` at `index.html:1080-1124`, including the month-name string, the Total and Per lesson strings, and the copy-msg-btn label. Do not change which months are selected for rendering at `index.html:1063-1066`, so the current calendar month still always gets a row. Do not change the three localStorage key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not change the group-card sort at `index.html:1029-1033`. No new dependencies, no build step. Out of scope: collapsing past months and scrolling the current month into view, which are separate larger changes. Acceptance criteria: With a group holding lessons in three or more distinct months, the first `.month-override-row` in the DOM is the latest month key and the last is the earliest. The current calendar month's row is within the first two rows whenever the current month has lessons or is the visible calendar month. Every month that rendered before the change still renders after it, with the same lesson count, the same Total and the same Per lesson value. Clicking a month row still opens the schedule editor on that month via

`App.handlers.startEditingDates`. The per-month price input still updates only its own row's total while the schedule editor is open.

Verification: Serve the file locally, seed `groupLessonPlannerData` with one group whose `monthlyOverrides` contains at least the keys 2024-09, 2025-03 and 2026-08 each with dates, reload, open the group, then read the text of every `.month-override-row .month-name` in DOM order and assert the sequence is August 2026, March 2025, September 2024. Then assert the number of rows equals the number before the change for the same seed.

PROMPT QW-15: Ask the browser for persistent storage on load

Context: Repo lesson-planner, `index.html`. The app stores everything in `localStorage` and has never requested persistent storage. Measured on the live origin in Chromium on 2026-08-20: `navigator.storage.persisted()` returned false and `navigator.storage.persist` is available.

Browsers evict best-effort origins under storage pressure and WebKit additionally clears an origin's script-writable storage after seven days of browser use without user interaction on the site; origins granted persistence are exempt from both.

Task: In `App.init()` at `index.html:409–422`, after the storage load call at `index.html:411`, add a guarded call that requests persistent storage when the API is available and persistence has not already been granted. The call must be fully feature-detected, must not await anything on the critical rendering path, and must never throw into `init()`. Record the outcome on the existing `App.state` object at `index.html:363–373` as a new property named `storagePersisted`, initialised to null and set to the boolean result once known, so a test can read it back. Do not surface it in the UI.

Constraints: Must not block or delay rendering, and must not change the order or the synchronous behaviour of any existing statement in `App.init()`. Must be a no-op with no console error when `navigator.storage` or `navigator.storage.persist` is absent, leaving `App.state.storagePersisted` as null. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not add a permission prompt of your own and do not react to a denial with a dialog; a denial is an acceptable outcome. No new dependencies, no build step.

Acceptance criteria: On a fresh load over HTTPS or `http://127.0.0.1` in a Chromium browser, `App.state.storagePersisted` settles to either true or false within one second of load, and the console error list is empty. When `navigator.storage.persisted()` already returns true before load, `navigator.storage.persist` is never invoked. When `navigator.storage` is undefined, the page still loads, the group list and month dropdown render, + Add Group opens the modal, `App.state.storagePersisted` remains null, and the console error list is empty. The three existing `localStorage` keys and their contents are byte-identical before and after the change for the same user actions.

Verification: Serve locally over `http://127.0.0.1` and load the page; read `App.state.storagePersisted` and assert it is a boolean, and assert the console error list is empty. Wrap `navigator.storage.persist` in a counting spy installed before load and assert it was called zero times when `navigator.storage.persisted()` already resolves true. Then delete `navigator.storage` from the window before load and assert `App.state.storagePersisted` is null, the group list renders, and the console error list is empty.

PROMPT QW-16: Let the calendar control row wrap, and stop the calendar summary clipping its text

Context: Repo `lesson-planner`, `index.html`. The `.calendar-controls` rule at `index.html:179` sets `display:flex` with `justify-content:center` and no `flex-wrap`, so it computes `flex-wrap: nowrap`. Measured at a 375x812 viewport with the schedule editor open: the control row has `scrollWidth` 364 against `clientWidth` 315; the previous-month button at `index.html:318` renders at `x = -19.1` with width 35.3, leaving 16.2px on screen; Clear Month at `index.html:323` extends to `x = 394.1` in a 375px viewport; and the enclosing `.modal` has `scrollWidth` 394 against `clientWidth` 375. Setting the modal's `scrollLeft` to -9999 leaves it at 0, so the clipped left portion of the previous-month button cannot be brought into view by scrolling at all – it is the only permanently unreachable control in the application. Separately, the `#calendar-summary` rule at `index.html:194–200` sets `height: 1em`, which measures `clientHeight` 12 against `scrollHeight` 14 at both 375px and 768px, clipping the text.

Task: Add `flex-wrap: wrap` to the `.calendar-controls` rule at `index.html:179` so the six controls wrap onto additional rows at narrow widths instead of overflowing. In the `#calendar-summary` rule at `index.html:194–200`, replace `height: 1em` with `min-height: 1em` so the summary can grow to fit its own text.

Constraints: Do not change the markup order of the six calendar controls at `index.html:317–324`, and do not change their ids. Keep `justify-content: center`. Do not change the calendar grid rules at `index.html:175–178` or the `.calendar-dow` rules at `index.html:181–190`. Do not change the calendar summary's text content, its em dash, its font-size, colour or text-align. Do not change the three `localStorage` key names or the persisted data shape. No new dependencies and no media queries; a single `flex-wrap` declaration is sufficient. Out of scope: increasing touch target sizes, which is tracked separately.

Acceptance criteria: At a 375px viewport with the schedule editor open, every one of the six calendar controls has a bounding rectangle whose left edge is at or above 0 and whose right edge is at or below the viewport width. The enclosing `.modal` reports `scrollWidth` equal to `clientWidth`, so no horizontal scrollbar appears inside the dialog. At a 1280px viewport the control row still renders on a single centred line with the same visual order. `#calendar-summary` reports `scrollHeight` equal to `clientHeight` at 375px, 768px and 1280px, and its full text including the em dash is visible.

Verification: Serve locally, seed one group with dates, open the group, open the schedule editor, set the viewport to 375x812 and read `getBoundingClientRect` for each of `prevMonthBtn`, `monthSelect`, `yearInput`, `nextMonthBtn`, `todayBtn` and `clearMonthBtn`, asserting the left edge is at least 0 and the right edge at most 375. Assert the `.modal` `scrollWidth` equals its `clientWidth`. Then select one date and assert `#calendar-summary` `scrollHeight` equals `clientHeight`. Repeat the summary assertion at 768x1024 and 1280x900, and confirm the control row is a single line at 1280px.

PROMPT QW-17: Clear the group-name prefill and replace it with a non-identifying placeholder
Context: Repo lesson-planner, `index.html`. `App.handlers.openGroupModal` at `index.html:603-637` sets `groupNameInput.value` to the literal string `Untitled Group` at `index.html:614` in add mode, then focuses that input at `index.html:635` after a 100ms timeout. Runtime inspection confirmed the input's `selectionStart` and `selectionEnd` are both 14 when focus lands, so nothing is selected and the caret sits at the end; typing `B2 Adults` immediately after opening the dialog produced the value `Untitled GroupB2 Adults`. Every new group therefore requires deleting 14 characters first. Separately, `index.html` contains zero placeholder attributes on any input, and the free-text group name is the only field in the entire data model that can turn a schedule record into personal data about a student – adopting non-identifying group labels is the highest-leverage, zero-cost data-minimisation measure available. These two findings resolve to a single change on a single attribute, so the placeholder value is chosen to serve the minimisation goal rather than to restate the fallback name.

Task: In add mode, stop prefilling the group name with a literal value: set `groupNameInput.value` to an empty string at `index.html:614`. Add a placeholder attribute to the input at `index.html:282` whose value is a schedule-style label example combining a weekday, a time and a level, in plain ASCII English consistent with the rest of the chrome, which `index.html:2` declares as `lang="en"`. Leave the focus call at `index.html:635` as it is.

Constraints: Do not use the string `Untitled Group` as the placeholder value; the placeholder's job is to demonstrate a non-identifying label. Do not change `App.handlers.saveGroup`'s empty-name fallback at `index.html:661`, which must still produce `Untitled Group` when the field is left blank. Do not change the different fallback `Untitled` at `index.html:677` used on edit. Do not change edit mode, where `groupNameInput.value` is correctly set from the existing group at `index.html:623`. Note that `App.render.groupInfo` at `index.html:997` and `1008` reads `groupNameInput.value` when no group exists yet and already falls back to `Untitled Group`, so verify the add-mode display still shows a sensible name with the field empty. Do not add a `maxlength`, required, pattern or any other validation attribute. Do not add a placeholder to any other input in this change. Do not change the three `localStorage` key names or the persisted data shape. No new dependencies and no build step.

Acceptance criteria: Clicking + Add Group leaves the group name field empty with a visible greyed placeholder showing a schedule-style example. Typing `B2 Adults` immediately after the dialog opens produces exactly the value `B2 Adults`. Clicking Save with the field left untouched creates a group named `Untitled Group`. The read-only group name display shown while adding renders neither an empty string nor the word `undefined`. Opening an existing group still prefills its real name, shows no hint, and still allows editing it. The placeholder contains no personal name and no digits other than a clock time. Exactly one placeholder attribute exists in `index.html` after the change.

Verification: Serve locally with empty `localStorage`, click + Add Group, assert `groupNameInput.value` is the empty string and read its placeholder attribute, then type `B2 Adults` and assert the value is exactly `B2 Adults`. Reload, click + Add Group, click Save without typing, and assert the stored group name is `Untitled Group`. Reload again, open the

existing group, and assert `groupNameInput.value` equals the stored name and no placeholder is displayed. Finally grep `index.html` for placeholder and assert exactly one match.

PROMPT QW-18: Refresh the group list after a schedule save

Context: Repo `lesson-planner`, `index.html`. `saveDateChanges` at `index.html:745-772` persists the new dates and monthly overrides and then calls `App.render.monthlyOverrides()`, `App.render.groupInfo()` and `App.render.calendar()`, but never `App.render.groups()`. The group card's lesson count is rendered from `group.dates.length` at `index.html:1045` by `render.groups`. Verified consequence: after selecting four dates and pressing Done, the card behind the modal still reads 0 planned lessons while the correct five dates are already persisted; a page reload shows the right number. The user is shown a wrong count and may redo work.

Task: Add a call to `App.render.groups()` inside `saveDateChanges`, after `App.services.storage.save()` at `index.html:764`, so the card count reflects the freshly saved dates.

Constraints: Do not change the three `localStorage` key names or the persisted data shape. Do not reorder or remove the existing render calls, and do not change `resetEditingContext`. Keep the change to a single added call; do not refactor `saveDateChanges`. Note that `render.groups` rebuilds the whole list with `innerHTML` and reattaches card click handlers, so no handler rewiring is needed. No new dependencies.

Acceptance criteria: Open a group with zero lessons, select four dates in the visible month, press Done, and the group card immediately reads 4 planned lessons without a reload.

Reloading the page shows the same count. Pressing Cancel instead of Done leaves the card count unchanged. Deleting a group still empties the list correctly.

Verification: Serve locally, add a group, open it, open the schedule editor, click a weekday header to select four dates, press Done, then read the `.group-card` text content and assert it contains 4 planned lessons. Reload and assert the same text.

PROMPT QW-19: Name each copy button after the month it copies

Context: Repo `lesson-planner`, `index.html`. `App.render.createMonthRow` at `index.html:1080-1124` renders one button per month row with the literal label of a clipboard emoji followed by Copy Payment Message at `index.html:1099`. The month name is in a sibling div at `index.html:1089` and is not programmatically associated with the button. Measured on a seeded dataset of one group with 36 months of lessons: 36 buttons in the same dialog share one distinct accessible name. The same document also exposes four buttons named Cancel and two named Save. This makes role-and-name queries ambiguous for the planned Playwright suite, and gives a screen-reader user 36 indistinguishable controls.

Task: In `App.render.createMonthRow`, add an `aria-label` to the copy button at `index.html:1099` that includes the month name and year already computed at `index.html:1084` and `1083`, for example a label of the form Copy payment message for August 2026. Keep the visible button text exactly as it is.

Constraints: Do not change the visible label text, the `copy-msg-btn` class name, or the disabled attribute logic that disables the button when the lesson count is 0. Do not change the month-name string at `index.html:1089` or the Total and Per lesson strings. Do not change the click handler at `index.html:1118-1122` or the message generated by

`App.utils.generateMonthlyPaymentMessage`. Do not change the three `localStorage` key names or the persisted data shape. No new dependencies. Note that the month name comes from `App.config.monthNames` at `index.html:379` and must not be re-derived some other way.

Acceptance criteria: With a group holding lessons in three or more distinct months, every `.copy-msg-btn` in the dialog has a distinct accessible name, and each name contains that row's month name and four-digit year exactly as the row's own heading shows them. The visible text of every such button still contains Copy Payment Message. A month with zero lessons still renders a disabled button. Clicking any button still opens the review modal with the message for that month and no other.

Verification: Serve locally, seed one group with `monthlyOverrides` for at least 2024-09, 2025-03 and 2026-08 each holding dates, reload, open the group, then collect the `aria-label` of

every `.copy-msg-btn` and assert the set has the same size as the number of rows and that each label contains the matching month name and year. Assert every button's `textContent` still contains Copy Payment Message. Click the button in the 2025-03 row and assert the review `textarea` content corresponds to that month's lesson count.

PROMPT QW-20: Fix pluralisation in the lesson and day count strings

Context: Repo `lesson-planner`, `index.html`. Two user-facing strings hardcode a plural noun. The month row at `index.html:1089` renders a parenthesised lesson count that produces (1 lessons). The calendar summary at `index.html:1172` renders a selected-day count that produces 1 days selected in September. Both were observed in a running browser. These strings are also intended test anchors, so their exact wording should be settled before an automated suite asserts on them. The identical grammatical defect in the shipped default payment template is handled by the `personal-data` prompt, which edits the same string literal, and is therefore out of scope here.

Task: Introduce a small helper in `App.utils` that returns a correctly pluralised noun for a count, and use it for the lessons noun at `index.html:1089` and the days noun at `index.html:1172` so a count of 1 renders lesson and day while all other counts render lessons and days.

Constraints: Change only the noun. Keep the rest of each string byte-identical, including the parentheses around the month row count, the words selected in, the literal `Total:` prefix and the em dash character in the calendar summary. Do not change the group card string that renders the planned-lessons count at `index.html:1049`, which is out of scope for this prompt. Do not change `App.config.defaultTemplate`; its wording is fixed by a separate prompt. Do not change the three `localStorage` key names or the persisted data shape. No new dependencies and no internationalisation library; English-only pluralisation is sufficient because all formatting is already pinned to `en-US`. Note that two existing specs in the uncommitted Playwright suite assert the current buggy strings, so retag and update those specs in the same change or immediately before it.

Acceptance criteria: A month with exactly one lesson renders (1 lesson) and a month with four renders (4 lessons). A selection of exactly one date renders a summary reading 1 day selected followed by the month name, an em dash, and the total, with the em dash preserved. Two or more dates render N days selected. A selection of zero dates still renders an empty calendar summary. No spec in the suite asserts the old plural forms.

Verification: Serve locally, open a group, open the schedule editor, select exactly one date and read the text of `#calendar-summary` and of the `.month-override-row` month name; then select a second date and read both again. Assert singular and plural forms respectively, and assert the summary still contains the em dash character. Then `grep` the spec files for the old strings and confirm zero matches.

PROMPT QW-21: Land the diverged working copy of `index.html` and freeze its test-hook set

Context: Repo `lesson-planner`. The committed and deployed `index.html` is 1473 lines and 58,649 bytes. A modified working copy exists uncommitted in the developer's main checkout at 1491 lines and 59,604 bytes. Two independent reads agree the diff does exactly two things. First, it adds the `hidden` attribute to all three modal overlays at `index.html:246`, 259 and 348, adds a CSS rule making `[hidden]` display none with importance next to the existing `.hidden` rule at `index.html:143`, and toggles `hidden` in all six modal open and close handlers. Second, it adds eight distinct `data-testid` values, being `group-card-name`, `group-card-lesson-count`, `month-name`, `month-lesson-count`, `month-total`, `month-price-input`, `price-per-lesson` and `copy-payment-message`, plus six `data-*` dataset attributes, being `data-group-name`, `data-group-index`, `data-month-key`, `data-weekday`, `data-date` and `data-day`. The first half closes a verified defect: on a loaded page 9 of 14 tab stops sit inside closed invisible modals, and activating them throws an uncaught page error from `index.html:558`, writes an empty string to `paymentTemplate`, and silently overwrites the clipboard. The second half is what makes the existing uncommitted Playwright suite able to locate anything at all, because its page objects locate group cards by `data-group-name` and month rows by `data-month-key`, neither of which exists in the deployed

file.

Task: Bring that diff onto the committed index.html so exactly one version of the file exists, committed and deployed. Then record the eight data-testid values and the six data-* attribute names in a new committed markdown file under docs/ as a frozen contract, stating that any future React implementation must emit the same data-testid values on the same semantic elements. This prompt creates that contract file.

Constraints: The working copy is a copy of the same original file and therefore still contains the personal payment details, so this is a real merge hazard and not a theoretical one: the template block must remain in the state left by the personal-data prompt, and check-no-personal-data.sh must exit zero after the merge. Enumerate the diff before applying it and reject anything beyond the modal-visibility fix and the fourteen attributes. Keep the .show class and the opacity transition so the fade-in is preserved. Keep the Escape handler at index.html:534-539 working, including its detection of the open modal via classList.contains('show'). Do not add roles, aria-modal, a focus trap or focus restoration in this change; those belong to the migration. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not modify AGENTS.md or LICENSE. No new dependencies, no build step. Acceptance criteria: Exactly one index.html exists, and the bytes served by the live site match the committed file. check-no-personal-data.sh exits zero after the merge. On a freshly loaded page with one existing group and no modal open, tabbing from the top reaches exactly the five toolbar buttons and then leaves the document, with zero stops inside any modal. Closed overlays report a computed display of none. Opening each of the three modals still shows it, its controls are focusable while it is open, Escape still closes an open modal, and no uncaught error appears in the console. All eight data-testid values and all six data-* attributes are present in the committed file, and the new contract file lists all fourteen. Every acceptance criterion of every quick win already shipped before this one still holds. Verification: Diff the two files and confirm the change set is exactly the modal-visibility fix plus the fourteen attributes, with no other edit. Run check-no-personal-data.sh and confirm exit zero. Serve locally, seed one group, tab from the document start recording each active element id, and assert the sequence is addGroupBtn, editTemplateBtn, loadCsvBtn, saveCsvBtn, clearDataBtn and then no modal control. Open and close each modal once. After deploying, fetch the live URL and byte-compare it against the committed file. Finally grep the committed file for each of the fourteen attribute names and confirm all are found.

PROMPT QW-22: Commit the toolchain files that AGENTS.md already documents

Context: Repo lesson-planner. On the main branch the committed application tree is only index.html, AGENTS.md, LICENSE and .gitignore, plus a docs/research tree added by the research programme whose docs/research/tools/package.json belongs to the report build pipeline and is unrelated to the app. There is NO root package.json, so AGENTS.md lines 9 to 15 document npm install, npm run serve on port 4173, npm run test:e2e, npm run test:ui, npm run test:update and npm run test:trace, and AGENTS.md line 5 documents an e2e/ directory, none of which is runnable today. They DO exist as uncommitted files in the developer's main checkout: package.json with devDependencies for faker, Playwright, http-server and TypeScript plus those exact scripts, playwright.config.ts with testDir e2e/features, a baseUrl on port 4173, timezoneId UTC and a webServer block, tsconfig.json covering e2e/**/*.ts, and an e2e/ suite of 38 TypeScript files.

Task: Commit package.json, package-lock.json, playwright.config.ts, tsconfig.json and the e2e/ directory so that the commands AGENTS.md documents actually work from a fresh clone. Generate package-lock.json by running npm install once. Do not change any script name, the port 4173 or the testDir, because AGENTS.md line 38 requires npm run serve and playwright.config.ts to stay aligned.

Constraints: Do not modify index.html. Do not modify AGENTS.md. Do not modify the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape; this prompt adds tooling files only and must not touch application code. Do not commit node_modules, test-results or playwright-report, all of which .gitignore already covers. Do not add React, Vite, a bundler,

a linter or any dependency that is not already present in the uncommitted package.json. If the diverged index.html has already landed, do not bring across any further change to that file in this commit.

Acceptance criteria: A fresh clone followed by `npm ci` then `npm run test:e2e` runs the suite without a missing-file or missing-script error. `git status` is clean after the commit apart from ignored paths. `index.html` is byte-identical to its state immediately before this commit. `npm run serve` starts a server on port 4173.

Verification: Record the byte count of `index.html` before and after and assert they match. Run `npm ci` then `npm run test:e2e` and record the pass and fail counts. Run `npm run serve` and confirm the port responds. Run `git status --porcelain` and confirm no unexpected untracked file.

PROMPT QW-23: Make group cards operable from the keyboard

Context: Repo lesson-planner, `index.html`. `App.render.createGroupCard` at `index.html:1041-1053` builds each card as a plain div with class `group-card` and an onclick handler that calls `App.handlers.openGroupModal(index)`. Runtime inspection confirmed the card computes `tabIndex -1`, has no role attribute and has no key handler. A full 16-step Tab sweep on a page with one card present never reaches the card, so once the group modal has been closed there is no keyboard route to open any group again. This is the return-visit entry point to the entire product.

Task: In `App.render.createGroupCard`, make the card keyboard operable by adding `tabindex="0"`, `role="button"` and a keydown handler that calls the same `App.handlers.openGroupModal(index)` when the pressed key is Enter or the space character, calling `preventDefault` for the space case so the page does not scroll. Give the card an accessible name that includes the group name.

Constraints: Do not change the card's visual appearance, its class name `group-card`, or the `.group-card` CSS at `index.html:23-46`. Do not change the existing onclick handler; the keydown handler is additive so mouse behaviour is untouched. Do not change the `h2` and `.group-card-info` markup inside the card at `index.html:1046-1051`, and do not change the planned-lessons wording. If the group-name escaping prompt has already landed, the accessible name must use the escaped value or be set as a property rather than interpolated into markup, so this change does not reopen the injection sink. Do not change the three `localStorage` key names or the persisted data shape. Do not add a focus-ring style in this change; the browser default is acceptable here and the palette work is tracked separately. No new dependencies.

Acceptance criteria: On a page with three group cards and no modal open, tabbing from the document start reaches all three cards in the order they appear in the DOM, before or after the toolbar buttons but never skipped. Pressing Enter on a focused card opens the group modal for that specific card's group. Pressing the space bar on a focused card opens the same modal and does not scroll the page. Clicking a card with the mouse still opens the correct group. Each card exposes an accessible name that contains its group name. Deleting a group and re-rendering still produces focusable cards for the remaining groups. A group whose name contains markup still renders that markup as literal text and creates no element.

Verification: Serve locally, seed three groups with distinct names, reload, then Tab from the document start recording the active element for each stop and assert all three `.group-card` elements appear. Focus the second card, press Enter, and assert `App.state.editingIndex` matches that card's group and the modal has the `show` class. Close the modal, focus the third card, press the space bar, and assert the correct group opened and `window.scrollTo` is unchanged. Click the first card with the mouse and assert it still opens. Finally seed a fourth group whose name contains an `img` tag with an `onerror` attribute and assert no `IMG` element exists in the document.

PROMPT QW-24: Extend `.gitignore` to cover build output, environment files and credential material

Context: Repo lesson-planner. The committed `.gitignore` is six lines and covers only Node and Playwright artifacts: `node_modules`, `playwright-report`, `test-results`, `e2e/.cache` and

.playwright-cache. Two separate gaps exist and are closed together because they are one edit to one file. First, AGENTS.md line 36 states that generated output must never be committed, and no bundler exists yet, so none of the directories a bundled build will produce are ignored: a Vite build defaults its output to dist, and the deployment plan for this repository introduces publish and preview-root as assembled output directories. Second, a static site has nowhere to hold a secret, values prefixed for client exposure are embedded in the bundle at build time, and GitHub secret scanning runs automatically on public repositories and reports detected partner secrets to the provider. This repository is public and is the deployment source for the live site, so any credential committed here is published. No tooling that would create env files exists yet, which is exactly why the ignore rules should land before it does.

Task: Append two clearly commented sections to .gitignore below the existing entries. The first covers build output: dist, dist-ssr, publish, preview-root, the Vite cache directory .vite, and *.local, which is the convention Vite's own template gitignore uses for local-only files. The second covers environment files and credential material: .env and every .env.* variant, while explicitly un-ignoring a committed example file named .env.example; service-account and credential JSON files matched by the patterns *-service-account*.json, service-account*.json and google-credentials*.json; and *.pem plus *.p12. State in the added comments that .gitignore only affects untracked files and does not remove anything already committed. Constraints: Do not remove, reorder or reword any existing line in .gitignore. Do not create any of the ignored files, and do not create .env.example itself; the negation exists so a future example file can be committed deliberately. Do not modify index.html, AGENTS.md or LICENSE. Do not add a package.json. Do not add editor-specific or OS-specific entries in this change. Do not run any git history rewriting. Ensure the *.local pattern and the .env.example negation do not conflict in either order.

Acceptance criteria: .gitignore still contains all five original entries in their original order. git check-ignore -v reports a matching rule for each of dist/index.html, publish/index.html, preview-root/x, .vite/deps, foo.local, .env, .env.local, .env.production, credentials.pem, key.p12 and my-service-account.json. git check-ignore returns no match for .env.example, exiting non-zero. Creating an empty directory named dist with a file inside it leaves git status --porcelain showing nothing for that path, and the same holds for publish and preview-root. The original rules still match node_modules and .playwright-cache. No tracked file is removed from the index by this change, and .gitignore is the only modified path.

Verification: From the repository root run git check-ignore -v on each of the eleven paths listed above and confirm every one reports a matching rule and line number. Run git check-ignore -v .env.example and confirm it reports no match. Run git check-ignore -v on node_modules and .playwright-cache and confirm the original rules still match. Run git status --porcelain and confirm .gitignore is the only change. Finally run git ls-files and confirm the tracked file list is unchanged.

PROMPT QW-25: Copy the DOM-free domain logic into typed modules with unit tests

Context: Repo lesson-planner. The deployed app is one file, index.html, with a single inline script and no build step; the repository root has no tsconfig.json for application code.

Statically verified: the whole of App.utils at index.html:1365-1467 and the whole of App.services.groups and App.services.csv at index.html:1214-1361 touch no DOM. Two of those functions read globals rather than parameters: App.utils.generateMonthlyPaymentMessage at index.html:1367-1378 reads the template from localStorage via

App.services.storage.getTemplate() at index.html:1207-1209, and

App.services.groups.getActiveOverrides at index.html:1242-1245 reads the App.state.isEditing global. A third business rule lives in markup: the currency whitelist exists only as option values at index.html:292-294 and is read back out of the DOM at index.html:1001-1002.

Task: Create a new application source directory and the TypeScript configuration for it, and port the following into TypeScript modules as pure functions with explicit parameters and no globals: pad, iso, toMonthKey, startWeekday, escapeCsvValue, parseCsv, normalizeMonthKey, deepClone and formatCurrency from App.utils; cloneOverrides, collectDatesByMonth,

ensureOverride, normalizeOverrides and flattenSelectedDates from App.services.groups; serialize and deserialize from App.services.csv; and generateMonthlyPaymentMessage rewritten to take the template string and the overrides object as parameters instead of reading localStorage and App.state. Encode the currency whitelist as a TypeScript union type of the two values currently present as option values, rather than reading the DOM. Add unit tests covering each exported function, including the inputs known to misbehave: a one-digit year producing a date key whose year part is shorter than four characters, a currency value that is not exactly three ASCII letters, and a CSV containing a stray quote that happens to balance. Add Vite and Vitest as devDependencies and a test:unit script that runs vitest once. Constraints: Use Vitest, not the Node built-in test runner. The React build introduced later needs Vite anyway and Vitest reuses that config and transform pipeline, so choosing Vitest here avoids migrating the unit layer twice; this is a deliberate override of an earlier report's stated preference for node:test, which that report labelled a preference rather than a requirement. Do NOT modify index.html at all – this is a deliberate temporary duplication, and the deployed file must keep its zero-dependency, zero-build, single-file property until a later cutover. Do not modify AGENTS.md. If a root package.json already exists from the toolchain commit, extend it rather than replacing it, and do not rename or repoint any existing script. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape – the ported CSV and override functions must produce byte-identical output to the current implementation for the same input. Do not port App.utils.formatDate, which is verified never called. Do not port App.render, App.handlers or App.services.storage. Do not use the any type. Tests must not embed the default payment template from index.html:382-400 – use a neutral template containing only the three tokens.

Acceptance criteria: The test:unit script exists, runs, and every test passes. Every function named in the Task has a typed export and at least one test. serialize applied to a group fixture produces a string whose first row is the six quoted column headers in the existing order, whose rows end with CRLF, and which contains no UTF-8 byte order mark, matching the current return value of App.services.csv.serialize exactly; note that a byte order mark, if added later, is applied at the Blob layer in saveToCsv and not inside serialize, so it is out of scope here. deserialize applied to that same string returns a groups array deep-equal to the input fixture. normalizeMonthKey throws for a month value of 13 and for a value with a one-digit year. formatCurrency throws for a currency value that is not exactly three ASCII letters, and the currency union type does not permit such a value. No file outside the new directories and the configuration files is modified; git status shows index.html unchanged. A grep over every new file returns zero matches for the fixed strings IBAN and MFO case-insensitively and zero matches for any digit run of eight or more.

Verification: Run the test:unit script and confirm all tests pass. Run git status and confirm index.html, AGENTS.md and LICENSE are unmodified. Run the two greps named in the acceptance criteria and confirm both return nothing.

PROMPT QW-26: Add a CI workflow that typechecks and runs the suite, and deploys nothing
Context: Repo lesson-planner. There is no .github directory and no CI of any kind, so no check runs on push or pull request. GitHub Pages is configured as Deploy from a branch with source main and the repository root, so every push to main publishes the repository root directly to the live site with zero automated checks, and the end user depends on that page for real work with her data living only in her browser. This prompt creates one new file and depends on the toolchain commit having landed the package.json that defines the typecheck, serve and test:e2e scripts. It deliberately merges two earlier proposals: one specified the typecheck step and the artifact uploads, the other specified the read-only permissions block and the explicit no-deploy boundary. Both are included.

Task: Create a new file .github/workflows/ci.yml defining a single job named test on ubuntu-latest with timeout-minutes 20, triggered on push to main, on all pull_request events and on workflow_dispatch, with a top-level permissions block of contents: read only. Steps in order: checkout; set up Node pinned to major version 22 with npm caching enabled; npm ci; npm run typecheck; npx playwright install --with-deps chromium; npm run test:e2e; then an upload of

playwright-report/ as an artifact with retention-days 7 that runs unless the workflow was cancelled; then an upload of test-results/ as an artifact with retention-days 7 that runs only on failure. If a unit-test script exists, insert it between typecheck and the browser install.

Constraints: This workflow must not deploy anything and must not change the GitHub Pages publishing source or any repository setting. Do not add any Pages step, any pages or id-token permission, or any environment block. Do not modify index.html, AGENTS.md or .gitignore. Do not add a browser-binary cache step; caching browser binaries is documented as not recommended because restore time matches download time. Install chromium only, not all browsers. Do not add a Node version matrix and do not use a floating lts alias; pin one major. Do not add new npm dependencies. Confirm .gitignore already excludes test-results/, playwright-report/ and node_modules/ and add any that are missing. The action major versions are version-sensitive: verify checkout, setup-node and upload-artifact against each action's own Releases API immediately before pinning rather than copying versions from any tutorial, because the official documentation prose is known to lag the release tags by a major.

Acceptance criteria: The workflow file parses as valid YAML. A pull request triggers exactly one job named test. The job runs every step in the declared order and reaches both artifact steps according to their conditions. The job fails if any spec fails and succeeds if all pass. It finishes inside its 20-minute timeout. No step references a browser other than chromium. No GitHub Pages deployment is created by this workflow, and the live URL still returns HTTP 200 serving the same content as before, with an unchanged etag. No test is skipped, quarantined or marked fixme to obtain a green result.

Verification: Push a branch and open a pull request; confirm the workflow triggers and the job runs to completion, and that playwright-report is downloadable. Deliberately break one assertion, push, confirm the run goes red and that both artifacts are uploaded, then revert. Finally request the live URL headers and confirm HTTP 200 with the same etag value as before the change.

PROMPT QW-27: Seed localStorage by overriding the storageState option instead of building a context

Context: Repo lesson-planner. The Playwright suite's fixture module currently overrides Playwright's built-in context fixture and calls browser.newContext with a storageState object. Two consequences: config options declared under use are only partially applied to a manually created context, so video is not recorded even though the config sets video retain-on-failure; and the base-URL fixture reads the project's use.baseURL with a hardcoded fallback that silently seeds the wrong origin if the config baseURL ever changes. Note that a localhost origin and a 127.0.0.1 origin are different origins for localStorage purposes, and the application reads storage synchronously during module initialisation, so seeding must happen before navigation.

Task: Remove the custom context fixture and the custom page fixture, and instead override Playwright's built-in storageState option with a fixture that depends on the existing planner-state option. Inside that fixture, derive the origin by passing the project's baseURL through the URL constructor and reading its origin property, with no string fallback: if baseURL is absent, throw an explicit error naming the missing option. Keep the storage-state builder as the single place that knows the three key names, and change its signature to accept the derived origin. Keep the clipboard stub working by moving it onto the built-in context fixture as an override that calls addInitScript and then delegates, or by calling it from an automatic fixture; do not reconstruct the context. Standardise the config baseURL on http://127.0.0.1:4173 and keep the webServer port at 4173.

Constraints: Do not modify index.html. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape or the raw-string encoding of paymentTemplate. Do not change any spec file. Do not change the acceptDownloads or timezoneId behaviour that tests already rely on. Do not add new npm dependencies. Never place the contents of the application's default payment template into any fixture. Note for later, and do not implement it now: once the bundled build sets a base path, the baseURL gains a /lesson-planner/ path segment and becomes

`http://127.0.0.1:4173/lesson-planner/`. Deriving the origin from the URL constructor rather than from a literal is what makes that later change a one-line edit, so do not reintroduce any literal origin.

Acceptance criteria: No file under the test directory contains a call to `browser.newContext`. No file under the test directory contains a `localhost` literal. The typecheck script passes. Every spec runs with the same pass or fail result as before the change. On a deliberately failing test, the HTML report contains a trace, a screenshot and a video attachment. Seeding still works, demonstrated by a test that seeds one group and sees it rendered.

Verification: Run the typecheck script, then the e2e script, and compare the pass or fail set against a run recorded before the change. Then make one assertion fail deliberately, open the report, and confirm all three artifact types are attached to that test. Revert the deliberate failure.

PROMPT QW-28: Freeze the clock before navigation and derive all test dates from it
Context: Repo `lesson-planner`. The application reads `new Date()` while its inline script initialises, at `index.html:369-370`, and again in `today highlighting`, in the `Today` button, in the schedule editor's default month, in the current-month cutoff of `updateDefaultPrice`, and in the CSV export filename. The tests currently exercise no clock control at all. Separately, the month-context helper in the test-data support module uses a `faker date-soon` call with a 240-day window, so although `faker` is seeded the base instant is the real wall clock and the month under test drifts every day; one spec also computes a month key from a live `new Date()` at module scope.

Task: Add an exported constant `FIXED_NOW` to the test-data support module holding a single deliberate instant that is mid-month, on a weekday, in a 31-day month. Add an exported helper that takes a `Page`, calls `page.clock.setFixedTime(FIXED_NOW)` and then navigates to the application root, and use it as the single navigation entry point for every spec. Remove any custom page fixture's implicit navigation if a previous prompt has not already done so. Rewrite the month-context helper and any other month or date derivation to compute offsets from `FIXED_NOW` using explicit month arithmetic instead of `faker` dates, keeping the function signature and return shape unchanged so specs need no edits beyond navigation. Replace the module-scope `new Date()` in the `payment-messages` spec with a derivation from `FIXED_NOW`.

Constraints: Use `setFixedTime`, not `clock.install` and not `pauseAt`: the application depends on real timers elapsing, including a 1000 ms copied-label window at `index.html:909` and three 100 ms focus delays, and freezing them would hang the copy flow. The clock must be set before the first navigation, because the application reads the date during module initialisation. Do not do this inside an automatic fixture; the ordering between an automatic fixture and the lazily created page fixture is not documented and an ordering bug here fails silently. Do not modify `index.html`. Do not change the three `localStorage` key names or the persisted data shape. Do not introduce a fake-timer library; the clock API is built in. Do not add any fixed wait anywhere. Keep `faker` for non-asserted values.

Acceptance criteria: No file under the test directory calls a `faker` date function. No spec file constructs a bare `new Date()` to derive a month or year under test. Every spec navigates through the new helper. Running the suite twice on different calendar days produces identical selected months, identical month-row headings and identical calendar summary strings. The clipboard test still passes, proving timers still elapse. The typecheck script passes.

Verification: Run the e2e script and record the resolved month for one schedule test from the report. Then set the machine clock forward by 90 days, or run the suite again on a later date, and confirm the same month appears. Confirm the copy-payment-message test still passes, which proves the 1000 ms timer still fires.

PROMPT QW-29: Add a storage contract spec for the three `localStorage` keys

Context: Repo `lesson-planner`. This prompt adds a new spec file and a new fixture directory to the committed Playwright suite. The application persists everything in exactly three `localStorage` keys: `groupLessonPlannerData` holding a JSON array of groups, `groupLessonPlannerSettings` holding a JSON object with a `defaultCurrency` property, and

paymentTemplate holding a raw string that is not JSON. There is no schema version field in the original shape, no validation on load and no error handling. Each group has name, price, currency, dates and monthlyOverrides, where monthlyOverrides is keyed YYYY-MM and each value has price and dates, and every date string is stored twice, once in group.dates and once in the relevant override's dates array. No test currently asserts any of this directly. Note that data-testid and data-* attributes now DO exist in the committed file, because the diverged working copy has landed and its eight test ids are a frozen contract; this spec nevertheless avoids them by deliberate choice rather than by necessity, because the storage contract is the one assertion surface that survives a framework rewrite unchanged and it should not acquire a dependency on markup that will not.

Task: Create a new spec file for the storage contract and a new fixture directory containing at least three committed JSON fixture files: an empty state, a single group with four dates in one month, and a group spanning three months with three different override prices. For each fixture, write a test that seeds it, loads the application, and asserts the visible invariants that depend on it: the group card count, the planned-lessons text, each month row heading with its lesson count, and each month's Total and Per lesson strings. Then write a write-side test that performs one schedule change through the UI and reads all three keys back, asserting the parsed structure including that group.dates and the union of all override dates arrays contain the same set of date strings. Add one test that seeds paymentTemplate with a synthetic template containing all three tokens and asserts it is used in the generated message for two different groups, proving the template is global.

Constraints: This spec must not use the existing page objects, and must not use data-testid, data-* attributes, id selectors, class selectors or XPath, so that it remains valid against the React implementation without edit. Locate only by visible text, and read state only by evaluating against the three key names as literal strings. Do not modify index.html. Do not change the three localStorage key names and do not change the persisted data shape; this spec describes the existing contract and must not require a source change to pass. Remember that paymentTemplate is a raw string and must not be JSON-encoded when seeded. Never place any part of the application's default payment template into a fixture or an assertion; use synthetic template text only. Use explicit literal group names, not faker, for anything asserted. Compare parsed structures, not JSON strings. Do not add new npm dependencies.

Acceptance criteria: Three or more fixture JSON files are committed. Every fixture has at least one test that asserts its visible invariants using visible text only. The write-side test fails if either dates array diverges from the other. The template test proves the template applies across two groups. The spec contains zero occurrences of a test-id query, zero occurrences of the substring data-, zero selectors beginning with a hash character and zero selectors beginning with a dot. No fixture file or assertion contains an IBAN-shaped string, a digit run of eight or more characters, or any Cyrillic text other than deliberately synthetic content. The typecheck script passes and the new spec runs green against the committed index.html.

Verification: Run only the new spec against a locally served copy of the committed index.html and confirm it passes with no other spec involved. Then hand-edit one fixture so a single override date is missing from group.dates, confirm the write-side invariant assertion fails, and restore the fixture. Finally grep the new fixture files and spec for an IBAN shape and for any run of eight or more digits and confirm zero matches.

PROMPT QW-30: Assert exported CSV content and a full round trip, not just the filename

Context: Repo lesson-planner. The CSV import and export spec's export test currently asserts only that the suggested filename matches a lesson-planner prefix pattern, so the export could produce entirely wrong content and the test would still pass. CSV is one of only two file paths out of the application. The export format is verified as: a header row of six quoted column names in the order Name, Default Price, Currency, Month, Month Price, Dates, with every field quoted, CRLF row endings, no UTF-8 byte order mark, and ISO dates space-delimited inside one quoted field.

Task: Extend the export test to read the downloaded file's bytes using the Playwright download object, and assert the exact header row text, the CRLF row endings, the number of

data rows, and the values of all six columns for a seeded group with two months at different prices. Add a separate round-trip test that seeds a known group, exports, clears all data through the UI, imports the exact file that was just downloaded, and then asserts structural equality of the parsed contents of `groupLessonPlannerData` and `groupLessonPlannerSettings` against their values before the export. Record the current absence of a UTF-8 byte order mark as an explicit assertion on the first three bytes so that adding one later is a deliberate, visible change rather than a silent one.

Constraints: Do not modify `index.html`; this prompt only observes the current export format and does not change it. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not assert on the template, which the export is known not to include. Compare parsed structures rather than JSON strings so that key ordering is not asserted by accident. Use a seeded group name drawn from an explicit literal, not from faker, so a quote or comma in a generated name cannot make the assertion non-deterministic. Do not add new npm dependencies. Never use the contents of the application's default payment template in any fixture. Note that the import step will now show a confirmation dialog if the `confirm-before-import` prompt has landed, so the round-trip test must handle that dialog rather than assuming none appears.

Acceptance criteria: The export test fails if the header row text changes by a single character. The export test fails if row endings change from CRLF to LF. The round-trip test fails if any group name, default price, currency, month key, month price or date is lost or altered. The byte-order-mark assertion states the current state explicitly and names the defect it tracks in a comment. The typecheck script passes and the whole suite still runs.

Verification: Run the e2e script. To prove the header assertion is actually sensitive, change one character of the expected header string inside the test, confirm the test fails on that exact assertion, and revert the test. Do not edit `index.html` to test sensitivity. Then run the round-trip test and confirm it passes.

PROMPT QW-31: Approval-pin the generated payment message and the CSV export bytes

Context: Repo lesson-planner. The two outputs whose regression is silent and costly are the substituted payment message, measured at 446 characters for the default template, and the CSV export bytes, whose format is a header row of six quoted columns, CRLF row endings, every field quoted, dates space-delimited inside one quoted field, and no UTF-8 byte order mark. Both are produced by functions that are pure once their inputs are parameters: the message by `App.utils.generateMonthlyPaymentMessage` at `index.html:1367-1378` and the CSV by `App.services.csv.serialize` at `index.html:1251-1282`. This prompt is independently shippable in either of two ways: run it after the typed-module port and target those modules, or run it standalone by copying just those two functions into a test-local module first. It creates new test and fixture files and modifies no existing file.

Task: Add approval tests, also called golden-master tests, that render each of the two outputs for a small fixed fixture and compare the whole output against a committed approved file rather than asserting field by field. Cover: the substituted payment message for a group with one lesson and for a group with four lessons; the serialized CSV for a group with no months, a group with two months, and a group whose name contains a comma, a double quote and Cyrillic characters. Alongside each approved file, record in the test itself an explicit label stating whether that output is contract, meaning it must never change, or defect, meaning it is expected to change when the corresponding defect is fixed. Label the singular-count message and the CSV outputs contract. Label nothing defect unless it corresponds to a numbered defect from the inventory report, and name that defect in the label.

Constraints: The fixture template must be a neutral placeholder containing only the three tokens for month, lessons and total plus surrounding neutral text. It must NOT be the default template at `index.html:382-400`, which contains a real person's full name, bank IBAN and tax identification number, and none of those values may appear in any test, fixture, approved file or comment. Do not modify `index.html`. Do not modify the ported functions to make them easier to test. Do not introduce an approval-testing library; a committed expected-output file plus a whole-string comparison is sufficient. Currency formatting must stay pinned to

the existing en-US locale so approved files are stable across machines. Any test that touches the current date must use a fixed date, not the system clock.

Acceptance criteria: Every approved file is committed and every approval test passes on a second run with no file rewritten. Each approval test carries a contract-or-defect label naming the defect where applicable. Changing the price in a fixture by one unit makes exactly the affected approval test fail, proving the pins are load-bearing rather than vacuous. Reordering the CSV columns in the ported serializer makes the CSV approval tests fail. A grep over every new file returns zero matches for the fixed strings IBAN and MFO case-insensitively and zero matches for any digit run of eight or more.

Verification: Run the test script twice and confirm all tests pass both times and that git status shows no approved file modified by the run. Temporarily change one fixture price by one unit, re-run, confirm exactly the expected tests fail, then revert. Run the greps in the acceptance criteria and confirm both return nothing.

PROMPT QW-32: Stop the favicon 404 by referencing an icon at the project path

Context: Repo lesson-planner, single deployed file index.html served from the subpath <https://denyslystopadskyy.github.io/lesson-planner/>. Verified with grep: index.html contains zero link elements and no base element, so the browser falls back to requesting /favicon.ico from the domain root. This is a console error on every single page load, and both the domain root and the project path return 404 – measured on 2026-08-20, the domain root itself returns 404, so nothing at the domain root can ever satisfy that request. This is the same absolute-path-versus-subpath failure class as a misconfigured bundler base path.

Task: Add a small icon file to the repository root, named favicon.svg, containing a simple flat SVG icon consistent with the app's existing accent colour, which is defined as the CSS custom property --accent at index.html:8. Then add a single icon link element inside the head of index.html, after the title element at index.html:6, whose href is the RELATIVE path favicon.svg with no leading slash, so that it resolves against the project subpath rather than the domain root. Use type image/svg+xml.

Constraints: The href must be relative. A leading slash would request the domain root and reproduce the bug. Do not add a base element. Do not change the existing meta charset, meta viewport or title elements. Do not touch the style block, the script block, or any application logic. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not add any external URL, CDN reference or font: the app currently has zero third-party requests and that property must be preserved. Do not add an apple-touch-icon, a web app manifest or a service worker; those are out of scope.

Acceptance criteria: favicon.svg exists at the repository root and is under 4 KB. index.html gains exactly one new line inside head, and grep for a double-quoted slash immediately after href in that line returns no match. Loading the page from a subpath produces a network request for favicon.svg under that subpath, not under the server root, and the browser console shows zero errors. The page still renders identically and the file remains a single self-contained HTML document with exactly one script element and one style element.

Verification: Serve the repository so the app is reachable at a subpath, load it with developer tools open, and confirm the network panel shows favicon.svg requested from the subpath with status 200 and the console panel shows no entries. Count script and style elements in index.html and assert each returns 1. Confirm the favicon appears in the browser tab.

PROMPT QW-33: Make buttons and form controls inherit the page font family

Context: Repo lesson-planner, index.html. The body font stack is declared at index.html:9 as a system-first stack. The generic button rule at index.html:11 sets cursor, border, background, border-radius and padding but no font, and form controls have no font declaration either, so buttons compute to the user-agent default rather than the page stack. Two elements already opt in individually: the payment-message pre rule at index.html:125 and the textarea rule at index.html:166, both with font-family inherit. The result today is that every button

and every input, select and number field renders in a different typeface from the surrounding text.

Task: Add `font-family: inherit` to the shared control styling so buttons, inputs, selects and textareas use the page font stack. Prefer adding a single rule covering button, input, select and textarea next to the existing button rule at `index.html:11`, and remove the now-redundant `font-family inherit` from the textarea rule at `index.html:166` only if the new rule provably covers it.

Constraints: Change the font FAMILY only. Do not set font-size, font-weight, line-height or the font shorthand, because the measured target sizes and the 375 px layout assume the current computed text size and a size change would invalidate them. Do not change any padding, border, border-radius, background or colour. Do not change the `.icon-button` font-size at `index.html:132`, which sizes the pencil emoji. Do not change the three localStorage key names or the persisted data shape. No new dependencies, no build step, no JavaScript change, no markup change.

Acceptance criteria: The computed font-family of the add-group button, the save-group button, the month select, the year input, the group name input and the template textarea each equals the computed font-family of body. The computed font-size of the add-group button is unchanged from before the edit. At a 375 px viewport the main screen still has `scrollWidth` equal to `clientWidth`, and with the schedule editor open every control in the calendar control row still has a bounding rect inside the modal content box. The pencil icon button still renders at its existing size.

Verification: Serve the file locally and, before and after the edit, record the computed style for the six elements listed above plus body, asserting font-family now matches body and that font-size for the add-group button is identical between the two recordings. Then repeat the 375 px overflow checks from the calendar quick win and confirm they still pass.

PROMPT QW-34: Add a UTF-8 BOM to the CSV export so Cyrillic names survive Excel on Windows
Context: Repo lesson-planner, `index.html`. `saveToCsv` at `index.html:915-931` builds the CSV with `App.services.csv.serialize` and wraps it in a Blob with a `text/csv` charset `utf-8` type at `index.html:921`. Inspection of a real downloaded file confirmed the bytes begin with the characters of the first column name and carry no UTF-8 byte order mark. The target user has Cyrillic group names, and Excel on Windows misreads BOM-less UTF-8 CSV as a legacy code page, so those names appear as mojibake. Note that the CSV path is an interchange format and is explicitly not the application's backup path; the versioned JSON envelope is. This change is worth making anyway because she may open the file in a spreadsheet.

Task: Prepend the single UTF-8 BOM character `U+FEFF` to the Blob contents in `saveToCsv` at `index.html:921` so the downloaded file starts with the bytes `EF BB BF`, leaving `App.services.csv.serialize` itself unchanged. No importer change is required: the header lookup at `index.html:1289` lowercases a trimmed cell, and `String.prototype.trim` strips `U+FEFF`, so the first column still resolves. Confirm this with the round-trip check below rather than assuming it.

Constraints: Do not change the header row text, the CRLF row endings, the all-fields-quoted convention or the column order Name, Default Price, Currency, Month, Month Price, Dates. Do not change the download filename pattern at `index.html:923-926`. Do not change the three localStorage key names or the persisted data shape. No new dependencies. Note that the existing CSV export test asserts the absence of a byte order mark as a deliberate pin, so that assertion must be updated in the same change rather than left to fail. Out of scope: adding the template to the CSV export, which the JSON envelope already covers.

Acceptance criteria: A downloaded export begins with the three bytes `EF BB BF`. The rest of the file is byte-identical to the previous output for the same data. Importing that exact downloaded file restores the same groups, dates, per-month prices and currency, with no alert shown, which proves the required-column lookup still finds the name column despite the BOM. A group name containing Cyrillic characters round-trips unchanged through export and import. The CSV export spec's byte-order-mark assertion now asserts presence rather than absence and passes.

Verification: Serve locally, create a group whose name contains Cyrillic characters, select a

date, press Done, click Save CSV, then inspect the first bytes of the downloaded file with a hex dump and assert they are EF BB BF. Load that same file back with Load CSV and assert the group name, dates and price match, with no alert dialog. Then run the CSV export spec and confirm it passes.

10. Top risks and mitigations

#	Risk	Mechanism	Mitigation
K1	Abandonment mid-migration — the single largest risk to the whole plan	One part-time engineer stalls, leaving two half-apps and a live site nobody dares touch	The plan is ordered so that every stall point is a working live site plus an unfinished experiment. R0 to R2 deliver the entire data-safety benefit before any React code exists, so the most valuable half of the programme cannot be lost to a stall. The staging path means the root URL is never mid-migration
K2	The personal identifiers reach a cloud copy, a backup file or a committed test fixture	The default template reaches every user who never edited it, and it is carried by the JSON envelope, the Firestore document, every snapshot, and any approval test that uses the real template	Hard blocker, enforced three ways: R0 step 5 lands first; the committed grep guard fails the file if a shape reappears; and QW-21's acceptance criteria require the guard to pass after the diverged copy is merged, because that copy still contains the values. No upload, email or shared folder before R0 step 5
K3	The staging build destroys the live data	Same origin means <code>/lesson-planner/next/</code> shares <code>localStorage</code> with the live app, and an unfinished React build writing the three keys is a realistic data-loss event, not a theoretical one	Prefixed keys by default (C9); <code>scripts/assert-storage-keys.sh</code> fails the build if the cutover bundle does not reference all three unprefixed literals, and is skipped for staging builds; staging is seeded by importing an export, never by sharing keys; a fresh off-laptop export exists before the staging URL is opened on any machine
K4	Line-number drift silently invalidates the prompts	QW-21 shifts every citation in nine reports by up to 18 lines. A prompt run against the wrong line edits the wrong thing and may still appear to succeed	QW-21 is positioned after every other <code>index.html</code> quick win. R3 carries an explicit task to re-verify citations in every unshipped prompt, with a done-when. The standing warning sits directly under the §9 heading. Where a prompt is run out of order, locate by symbol name

#	Risk	Mechanism	Mitigation
K5	The Pages publishing-source flip is declined or goes wrong	It is a manual repository-owner action in the web UI that no script can perform, and the documentation does not say what happens to the live site during the switch	D3 is answered before R8 starts. Sequenced so the first Actions deployment publishes byte-identical content, making the flip content-neutral: if it works nothing changes, if it fails the developer is watching. Never select <code>main / /docs</code> , which would publish the research reports instead of the app
K6	Silent parity drift at R9	The parity checklist is manual, and a missed inventory row is exactly the silent regression the migration exists to prevent. R9 is also the only L step and the only one whose scope can grow unnoticed	The exit criterion is "every row has a disposition", not "it looks done", and the checklist is committed. The dual-project report diff makes the outcome boolean. Accessibility and capability work are explicitly bucketed so scope creep is visible as a bucket violation
K7	Sync propagates corruption everywhere, with nothing to roll back to	Spark has no scheduled backup, no PITR and no managed export. One bad write to <code>users/{uid}</code> syncs to every device	The snapshot layer is mandatory, not optional (C4): client-rotated documents, deterministic retention, restore via Procedure C. Validate <code>schemaVersion</code> and that <code>groups</code> is an array before assigning any remote document. The JSON envelope is an independent copy that predates any bad write
K8	A silent write failure the user cannot detect	Firestore's latency compensation notifies local listeners before data reaches the server, so an indicator wired to the call site would show success during a total outage	The indicator advances only on <code>hasPendingWrites</code> becoming false or <code>waitForPendingWrites()</code> resolving. Acceptance criterion 19 requires a test that asserts the indicator does not advance while offline. Web offline persistence is off by default, so a boot-time fingerprint compare and push is required, not optional
K9	She never presses Save backup, so no independent copy exists	The residual manual step cannot be removed: a page cannot write to an arbitrary path unattended, the file picker needs a user gesture and is not available in every engine	The staleness indicator with a warning past 14 days; the fact that the cloud layers make a missed file backup no longer a single point of failure; and the pull-forward trigger in C2 that promotes sync if the indicator crosses 14 days twice. Rehearse the restore once while nothing is broken

#	Risk	Mechanism	Mitigation
K10	Approving a defect into permanence	The documented failure mode of approval testing: a snapshot is updated without review, and the wrong behaviour becomes the expected one. Six known-wrong behaviours are queued for change	Every approved file carries a contract-or-defect label naming the defect id. Every spec carries an INV / DEF / CHAR tag, reviewed with the developer before R9, not during it. A test that passes on both projects but is tagged DEF is itself a finding
K11	The migration is judged on the wrong axis and looks like a regression	The bundled app will be heavier on the wire and slower to first paint than 13,610 gzipped bytes in one request. If performance is the stated benefit, the migration fails on its own terms	State the justification as maintainability, testability, and the removal of the XSS and unhandled-error defect classes. Never sell it as performance. Measure the bundle once and record the number rather than discovering it in an argument
K12	Capability work starts before the storage layer is safe	The improvement backlog is attractive and the data-safety work is invisible. Three backlog items add fields to the persisted shape and therefore need the version key first	The version key ships in R2, and the three field-adding items are explicitly sequenced behind it in R12. Bucket discipline: nothing from bucket (ii) or (iii) starts while any R0 to R2 item is open
K13	Free-tier terms change, or the account becomes unusable	The plan starts requiring a payment method, an inactivity policy appears, an employer administrator blocks third-party authorisation, or the project is deleted by mistake	The switching triggers are written down with the specific signal for each. The local path has no dependency to rot, and the JSON envelope is provider-independent. Note the honest position: the absence of a documented inactivity policy is the strongest statement the evidence supports, not a guarantee
K14	The restore procedures are never rehearsed and fail when needed	An unrehearsed restore is a hope, and the prior art already shows the failure mode of handing over a procedure whose buttons do not exist	R2's done-when includes rehearsing Procedure B end to end with her. Procedures A and C are rehearsed at R11. Do not hand over any procedure before the controls it names exist
K15	A pulled-forward decision reintroduces a resolved conflict	Under time pressure the easy moves are: ship sync into the legacy file, share the live keys with staging, or grow the DOM-level suite because the hooks now exist	Each is named as a resolved conflict with its reasoning in §2, so re-litigating it is at least visible. The two that carry data-loss risk — sync in the legacy file and shared staging keys — are also the two whose mitigations are machine-checked by the key gate

11. Sources

These are the external sources this report's own conclusions rest on, carried forward from the input reports' source tables. The eight prior reports cite roughly two hundred sources between them; re-listing them would pad rather than inform.

#	Title	URL	Accessed	Supports
S1	Firestore — Scheduled backups	https://firebase.google.com/docs/firestore/backups	2026-08-20	Scheduled backups require the Blaze plan, and a restore creates a new database rather than replacing the original. C4, K7 (via docs/research/rp07-data-migration-recovery/)
S2	Firestore — Use point-in-time recovery	https://firebase.google.com/docs/firestore/use-pitr	2026-08-20	PITR requires billing enabled, so it is unavailable on the free plan. C4, K7
S3	Firestore — Export and import data	https://firebase.google.com/docs/firestore/manage-data/export-import	2026-08-20	The managed export and import service requires the Blaze plan. C4, K7
S4	Firebase — Best practices for signInWithRedirect flows	https://firebase.google.com/docs/auth/web/redirect-best-practices	2026-08-20	Third-party storage blocking breaks the redirect flow for apps off the Firebase auth domain, already required on Firefox 109+ and Safari 16.1+; popup avoids it. C5, T3
S5	Firebase Help — Set a web app's OAuth redirect domains	https://support.google.com/firebase/answer/6400741	2026-08-20	Only localhost and the project's hosting domain are authorized by default, and whitelisting a domain allows requests from any URL and port of that domain. C5
S6	Firebase — Learn about using and managing API keys	https://firebase.google.com/docs/projects/api-keys	2026-08-20	Firebase-restricted keys need not be secrets, conditionally, because Security Rules and App Check do the authorization work; the page names exceptions. §1 bullet 4, R11

#	Title	URL	Accessed	Supports
S7	Firebase — Security Rules and Firebase Authentication	https://firebase.google.com/docs/rules/rules-and-auth	2026-08-20	The per-user ownership pattern binding <code>request.auth.uid</code> to the document id, and that <code>request.auth</code> is null when unauthenticated. R11, acceptance criterion 21
S8	Firebase Pricing	https://firebase.google.com/pricing	2026-08-20	Spark needs no payment method; daily free write, read and delete allowances against which the snapshot layer's cost is judged. C4, K13
S9	Firestore — Locations	https://firebase.google.com/docs/firestore/locations	2026-08-20	<code>europe-central2</code> is Warsaw, Poland, and a database location cannot be changed once provisioned. §1 bullet 3, R11
S10	Firestore — Access data offline	https://firebase.google.com/docs/firestore/manage-data/enable-offline	2026-08-20	Web offline persistence is disabled by default, so the pending-write queue is in memory. K8, acceptance criterion 20
S11	Firestore — Get realtime updates	https://firebase.google.com/docs/firestore/query-data/listen	2026-08-20	Latency compensation notifies local listeners before data reaches the backend; <code>metadata.hasPendingWrites</code> indicates unsent changes. K8, acceptance criterion 19
S12	Supabase — Production checklist	https://supabase.com/docs/guides/platform/going-into-prod	2026-08-20	Free Plan projects with low activity in a seven-day period may be paused, and Free Plan backups are not downloadable — why Supabase is excluded. C4 background
S13	GitHub Docs — Configuring a publishing source for your GitHub Pages site	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site	2026-08-20	The two publishing sources, the exact Settings path, and that branch publishing allows root or a docs folder. D3, K5, C11

#	Title	URL	Accessed	Supports
S14	GitHub Docs — Using custom workflows with GitHub Pages	https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages	2026-08-20	Required permissions, the <code>github-pages</code> environment, and the job dependency for artifact deployment. R8
S15	GitHub Docs — Securing your GitHub Pages site with HTTPS	https://docs.github.com/en/pages/getting-started-with-github-pages/securing-your-github-pages-site-with-https	2026-08-20	Pages sites are publicly available even if the repository is private, and should not carry sensitive data. §4 step 5, K2
S16	Vite — Deploying a Static Site	https://vite.dev/guide/static-deploy	2026-08-20	For a project site the base must be set to the repository path; the reference Pages workflow. §1 bullet 5, R8
S17	Vite — Shared Options	https://vite.dev/config/shared-options.html	2026-08-20	<code>base</code> defaults to <code>/</code> , and the documented forms carry a trailing slash. T17, R8
S18	React Blog — Sunsetting Create React App	https://react.dev/blog/2025/02/14/sunsetting-create-react-app	2026-08-20	CRA is deprecated for new apps, and Vite is among the recommended build tools. §1 bullet 5
S19	Playwright — Locators	https://playwright.dev/docs/locators	2026-08-20	The recommended locator priority, test ids as the most resilient but least user-facing option, and CSS or XPath chains as a bad practice. C3, acceptance criterion 11
S20	Playwright — Clock	https://playwright.dev/docs/clock	2026-08-20	Install the clock before navigating; <code>setFixedTime</code> keeps timers running. QW-28, acceptance criterion 12

#	Title	URL	Accessed	Supports
S21	Playwright — Visual comparisons	https://playwright.dev/docs/test-snapshots	2026-08-20	Snapshots are named per browser and per platform because rendering differs — the basis for dropping pixel visual regression. §6
S22	MDN — <code>Window.showSaveFileDialog()</code>	https://developer.mozilla.org/en-US/docs/Web/API/Window/showSaveFileDialog	2026-08-20	Requires transient user activation and is not Baseline, so an unattended file backup is impossible. K9, T6
S23	Chrome for Developers — Persistent permissions for the File System Access API	https://developer.chrome.com/blog/persistent-permissions-for-the-file-system-access-api	2026-08-20	Chrome 122's allow-on-every-visit grant gives indefinite access to a stored handle. T6
S24	Web Content Accessibility Guidelines (WCAG) 2.2	https://www.w3.org/TR/WCAG22/	2026-08-20	The criterion set adopted as the accessibility target, and the resolver for T13's open question about SC 2.5.8's conformance level. C6
S25	Web Content Accessibility Guidelines (WCAG) 2.1	https://www.w3.org/TR/WCAG21/	2026-08-20	SC 2.5.5 Target Size is Level AAA at 44x44 and is the only target-size criterion in 2.1; SC 2.5.8 is absent from 2.1. C6
S26	npm registry — vitest	https://registry.npmjs.org/vitest	2026-08-20	Vitest's supported Vite peer range, which is what makes it reuse the React build's config rather than adding a second toolchain. C8, QW-25
S27	MDN — <code>Window.localStorage</code>	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	2026-08-20	<code>localStorage</code> is scoped per origin and per protocol — the basis of the origin-and-keys invariant, the staging-key risk and the never-change-the-origin rule. C9, C10, K3

#	Title	URL	Accessed	Supports
S28	MDN — Storage quotas and eviction criteria	https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria	2026-08-20	Best-effort versus persistent mode, what <code>navigator.storage.persist()</code> changes, and the documented Web Storage cap whose unit basis is unstated. C15, QW-15, T26
S29	WebKit — Full Third-Party Cookie Blocking and More	https://webkit.org/blog/10218/full-third-party-cookie-blocking-and-more/	2026-08-20	The seven-day cap on script-writable storage and the home-screen exemption. T2, D10
S30	Google — Chrome Sync and your data	https://support.google.com/chrome/a/answer/13616205	2026-08-20	The synced data types exclude site storage, IndexedDB and cookies, so browser sync cannot return the lost data. §4 step 3
S31	Apple — Data Protection in Apple devices	https://support.apple.com/guide/security/data-protection-sece8608431d/web	2026-08-20	Erasing obliterates the keys in effaceable storage and renders user data cryptographically inaccessible. T11
S32	Test Double — Refactoring Legacy code with tests	https://github.com/testdouble/contributing-tests/wiki/Refactoring-Legacy-code-with-tests	2026-08-20	Characterization tests are deleted after the refactor because redundant coverage is problematic — the basis for deleting the legacy DOM specs at R13. C3
S33	Martin Fowler — Branch By Abstraction	https://martinfowler.com/bliki/BranchByAbstraction.html	2026-08-20	One abstraction with multiple coexisting implementations, and that the abstraction layer is temporary scaffolding. The three storage keys are that abstraction. C9, R8

#	Title	URL	Accessed	Supports
S34	Regulation (EU) 2016/679 — consolidated text	https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A02016R0679-20160504	2026-08-20	The four conditions cancelling the Art. 30(5) derogation, including processing that is not occasional, and Art. 5(1)(f) covering accidental loss — why one written paragraph is required and why the storage hardening is a security-of-processing measure. R0, D9